DOCUMENTATION ET COURS DU PROJET

Assistant RAG & Évaluation d'IA générative

Architecture complète, fonctionnement de chaque module, choix de conception et alternatives

Un assistant de questions-réponses sur la documentation officielle de dbt et d'Apache Airflow, construit pour être mesuré : recherche multilingue et reranking, réponses ancrées et citées, protocole d'évaluation statistiquement fiable, alignement du générateur par DPO sur une récompense vérifiable, tests et CI/CD.

RAG · Embeddings multilingues · Reranking · Ragas · Bootstrap · DPO · QLoRA · FastAPI · Docker · GitHub Actions

AUTEUR	Rayan
DÉPÔT	`github.com/rayanghomsi10/Assistant-RAG`
VERSION DÉCRITE	V2 — commit 001ad99
DATE	13 octobre 2026

Sommaire

PARTIE I · COMPRENDRE

1 Introduction 4
- 1.1 Le problème 4
- 1.2 Le projet en une page 4
- 1.3 Comment lire ce document 5

2 Notions fondamentales 6
- 2.1 Modèle de langage, hallucination et fenêtre de contexte 6
- 2.2 Le RAG : génération augmentée par la recherche 6
- 2.3 Embeddings et similarité cosinus 7
- 2.4 Index vectoriel et recherche approchée 7
- 2.5 Le chunking 7
- 2.6 Bi-encodeur et cross-encodeur : le reranking 8
- 2.7 Prompting et ancrage 8
- 2.8 Évaluer un système RAG 8
- 2.9 Statistiques : intervalle de confiance, bootstrap, test apparié 9
- 2.10 Alignement par préférences : RLHF, PPO, DPO 9
- 2.11 LoRA et QLoRA 10
- 2.12 NLI : vérifier qu'une phrase découle d'un texte 10

PARTIE II · ARCHITECTURE

3 Vue d'ensemble 12
- 3.1 Les quatre chaînes 12
- 3.2 Artefacts et flux de données 13
- 3.3 Organisation du dépôt 13
- 3.4 Stack technique 14
- 3.5 Configuration centrale 14

PARTIE III · LES MODULES EN DÉTAIL

4 Construction du corpus 17
- 4.1 Rôle et sources 17
- 4.2 Fonctionnement 17
- 4.3 Fonctions 18
- 4.4 Choix et alternatives 19

5 Chunking 20
- 5.1 Fonctionnement 20
- 5.2 Fonctions 21
- 5.3 Choix et alternatives 21

6 Embeddings et reranker 23
- 6.1 Fonctions 23
- 6.2 Choix et alternatives 23

7 Base vectorielle et recherche 25
- 7.1 Fonctionnement 25
- 7.2 Fonctions 25
- 7.3 Choix et alternatives 26

8 Chaîne RAG et vérifications 27
- 8.1 Le prompt 27
- 8.2 Fonctions de chain.py 27
- 8.3 Les vérifications déterministes 28

9 API 30
- 9.1 Points d'entrée 30

10 Évaluation 31

10.1 Le jeu d'évaluation 31
10.2 Les métriques 31
10.3 Le déroulement d'un run 32
10.4 Comparer deux runs 33
10.5 Suivi MLflow 33
10.6 Fonctions 33
10.7 Choix et alternatives 34

11 Alignement par DPO 35
11.1 Objectif et principe 35
11.2 Étape 1 : les questions d'entraînement 35
11.3 Étape 2 : la récompense et les paires 35
11.4 Étape 3 : l'entraînement 36
11.5 Étape 4 : retour dans Ollama et décision 37
11.6 Choix et alternatives 38

12 Tests 39

13 Conteneurisation et CI/CD 41
13.1 Docker 41
13.2 CI/CD 41

PARTIE IV · PRENDRE DU RECUL

14 Historique : de la V1 à la V2 44
14.1 Ce que mesurait la V1 44
14.2 Ce que la relecture a montré 44

15 Récapitulatif des décisions 46

16 Limites et points non mesurés 48
16.1 Ce qui n'a pas encore été mesuré 48
16.2 Limites de conception 48

17 Pistes d'évolution 49

18 Mode d'emploi 50
18.1 Installation 50
18.2 Construire et interroger 50
18.3 Évaluer 50
18.4 Entraîner par DPO 50

19 Résultats mesurés et état d'avancement 51
19.1 Mise en route : une configuration restée en V1 51
19.2 La baseline V2 51
19.3 Lecture des résultats 53
19.4 Analyse des échecs 54
19.5 Ce que l'exécution réelle a révélé 54
19.6 État du pipeline DPO 55
19.7 Prochaines étapes 56

20 Observabilité, interface et conversation 57
20.1 Observabilité avec Langfuse 57
20.2 Interface web de test 58
20.3 Mémoire de conversation 59
20.4 Compléments : mises en garde vérifiées (expérimental) 60
20.5 Robustesse du pipeline DPO 61

ANNEXES · GLOSSAIRE ET RÉVISION

A Glossaire 63

B Questions de révision 65

PARTIE I

Comprendre

Ce que fait le projet, le problème qu'il résout, et les notions nécessaires pour suivre le reste du document : modèles de langage, RAG, embeddings, recherche vectorielle, reranking, évaluation, statistiques et alignement par préférences.

1 Introduction

Ce document décrit le projet **Assistant RAG & Évaluation d'IA générative** dans son état actuel (version 2, commit `001ad99`, dépôt `rayanghomsi10/Assistant-RAG`). Il est écrit comme un cours : chaque composant est expliqué, justifié, et confronté aux alternatives qu'on aurait pu choisir.

1.1 Le problème

Un ingénieur data qui travaille avec dbt et Apache Airflow se pose en permanence des questions précises : « quelle est la matérialisation par défaut d'un modèle dbt ? », « pourquoi mes XComs disparaissent-ils au retry ? », « quand un Dag `@daily` est-il réellement déclenché ? ». Un modèle de langage généraliste répond à ces questions avec aplomb, mais pas toujours correctement : sa connaissance est figée à sa date d'entraînement, il ne sait pas quelle version de l'outil vous utilisez, et il peut inventer une option qui n'existe pas.

Le projet construit un assistant qui répond **uniquement à partir de la documentation officielle**, cite ses sources, et dit « je ne trouve pas cette information » quand la réponse n'est pas dans les documents. Surtout, il **mesure** si l'assistant y parvient, avec un protocole d'évaluation conçu pour distinguer une vraie amélioration du bruit statistique. Enfin, il propose une boucle d'amélioration du modèle générateur par **DPO**, fondée sur des critères vérifiables plutôt que sur l'avis d'un autre modèle.

1.2 Le projet en une page

Le système se décompose en quatre chaînes qui partagent les mêmes briques :

1. **Construction de la base de connaissances** : téléchargement des sources de 10 pages de documentation (5 dbt, 5 Airflow) à des versions figées, conversion en Markdown propre, découpage en passages (chunks) par sections, vectorisation, indexation dans ChromaDB.
2. **Inférence** : pour une question, recherche des 20 passages les plus proches, reclassement par un cross-encoder, conservation des 4 meilleurs, génération de la réponse par Llama 3.1 8B (via Ollama) avec un prompt qui impose l'ancrage, les citations et le refus explicite. Exposée par une API FastAPI.
3. **Évaluation** : 85 questions (73 dans le périmètre, 12 hors périmètre) avec des preuves recopiées du corpus ; métriques de retrieval et de génération calculées sans juge, métriques Ragas avec un juge distinct du générateur, intervalles de confiance par bootstrap, comparaison appariée entre deux versions, suivi dans MLflow.
4. **Alignement** : génération de questions d'entraînement, réponses produites par le modèle actuel, notation par une récompense vérifiable, constitution de paires de préférences, entraînement DPO avec QLoRA sur GPU (Colab ou Kaggle), réimport dans Ollama, réévaluation.

Autour : 52 tests automatisés qui tournent sans modèle ni GPU, une image Docker, et une CI/CD GitHub Actions qui vérifie le code, le jeu d'évaluation et la reproductibilité du corpus, puis publie l'image.

> **ÉTAT RÉEL DU PROJET**
>
> Le code de la V2 est complet et testé. **La baseline V2 est mesurée** (chapitre 19) ; observabilité, interface, mémoire de conversation et compléments ont été ajoutés ensuite (chapitre 20). Le pipeline DPO s'est arrêté à l'étape 2, à relancer. Les chiffres de la V1 (chapitre 14) ne sont pas comparables à ceux de la V2. Un chiffre non mesuré n'apparaît nulle part comme un résultat.

1.3 Comment lire ce document

La **partie I** pose les notions. Si tu les maîtrises, passe directement à la **partie II**, qui donne l'architecture d'ensemble et la configuration. La **partie III** détaille chaque module : son rôle, son fonctionnement, la liste de ses fonctions, les choix faits et leurs alternatives. La **partie IV** prend du recul : historique V1 → V2, tableau des décisions, limites, pistes d'évolution et mode d'emploi. Les annexes regroupent le glossaire, la référence des métriques et des questions de révision corrigées.

Trois types d'encadrés reviennent :

> **POURQUOI CE CHOIX**
>
> La justification d'une décision de conception.

> **CE QU'ON AURAIT PU FAIRE**
>
> Les alternatives crédibles, avec ce qu'elles auraient apporté et coûté.

> **ATTENTION**
>
> Un piège, une limite ou un point non vérifié.

2 Notions fondamentales

Ce chapitre est un cours condensé. Chaque notion est présentée au niveau nécessaire pour comprendre les choix du projet, avec un renvoi vers le module qui l'utilise.

2.1 Modèle de langage, hallucination et fenêtre de contexte

Un **grand modèle de langage** (LLM) est un réseau de neurones (un transformer) entraîné à prédire le jeton suivant d'un texte. Un **jeton** (token) est un morceau de mot : « matérialisation » fait plusieurs jetons, « dbt » un ou deux. Le modèle génère une réponse jeton par jeton, chaque jeton étant tiré d'une distribution de probabilité. La **température** règle cette distribution : à 0, on prend toujours le jeton le plus probable (sortie quasi déterministe) ; plus haut, on explore davantage (sortie variée). Le projet génère à température 0 pour l'inférence et l'évaluation, et à 0,8 pour produire des réponses variées lors de la constitution des paires DPO.

Une **hallucination** est une affirmation fluide et plausible mais fausse ou non étayée. Elle vient du fait que le modèle optimise la vraisemblance du texte, pas sa vérité. Elle est particulièrement fréquente sur les détails précis : noms de paramètres, valeurs par défaut, numéros de version.

La **fenêtre de contexte** est le nombre maximal de jetons que le modèle peut lire en une fois (prompt + réponse). Llama 3.1 accepte jusqu'à 128 000 jetons, mais **Ollama** n'en alloue par défaut que quelques milliers, selon la version. Au-delà, le début du prompt est tronqué *sans erreur*. Avec un prompt système, 4 passages de 400 jetons et la question, on approche de ces limites ; c'est pourquoi le projet fixe explicitement `num_ctx = 8192` (variable `OLLAMA_NUM_CTX`).

Quantification : stocker les poids du modèle sur moins de bits (4 au lieu de 16) pour réduire la mémoire. `llama3.1:8b` dans Ollama est quantifié au format GGUF `Q4_K_M` : environ 5 Go au lieu de 16, au prix d'une légère perte de qualité. C'est ce qui permet de faire tourner un modèle de 8 milliards de paramètres sur un GPU de 8 Go.

2.2 Le RAG : génération augmentée par la recherche

Le **RAG** (Retrieval-Augmented Generation) consiste à chercher, au moment de la question, les passages de documents pertinents et à les insérer dans le prompt. Le modèle ne répond plus « de mémoire » mais en lisant une source. Trois bénéfices : la connaissance est à jour (il suffit de mettre à jour les documents), la réponse est vérifiable (on peut citer le passage), et on peut demander au modèle de refuser quand les passages ne contiennent pas la réponse.

Un RAG a deux points de défaillance distincts, et toute la conception de l'évaluation en découle :

- **Le retrieval** peut ne pas ramener le bon passage. Le meilleur générateur du monde ne peut alors pas répondre correctement.
- **La génération** peut mal exploiter un bon passage : réponse incomplète, détail inventé, citation déformée, refus alors que la réponse était là.

Mesurer uniquement la réponse finale ne permet pas de savoir lequel des deux a échoué. Le projet mesure donc le retrieval seul (sans LLM), puis la génération.

2.3 Embeddings et similarité cosinus

Un **embedding** est un vecteur de nombres réels (384 dimensions pour le modèle utilisé) qui représente le sens d'un texte. Un modèle d'embeddings est entraîné pour que deux textes de sens proche aient des vecteurs proches. La proximité se mesure par la **similarité cosinus** :

$$\cos(u, v) = (u \cdot v) \,/\, (\|u\| \cdot \|v\|)$$

Elle vaut 1 pour deux vecteurs de même direction, 0 pour deux vecteurs orthogonaux. Si les vecteurs sont normalisés (norme 1), la similarité cosinus est simplement leur produit scalaire ; le projet normalise systématiquement (`normalize_embeddings=True`). ChromaDB, configurée en espace `cosine`, renvoie une *distance* égale à 1 − similarité ; le code la reconvertit en similarité.

Un modèle d'embeddings **multilingue** place une phrase française et sa traduction anglaise à des positions voisines. C'est indispensable ici : les questions sont en français, le corpus en anglais. Le modèle de la V1 (`all-MiniLM-L6-v2`) avait été entraîné sur de l'anglais ; la recherche cross-lingue fonctionnait en partie par chance, sur les mots partagés (`xcom_push`, « dbt »). La V2 utilise `intfloat/multilingual-e5-small`.

Les modèles de la famille **E5** ont été entraînés avec des préfixes : `"query: "` devant les questions, `"passage: "` devant les documents. Les omettre dégrade nettement leurs performances. C'est une asymétrie volontaire : une question et le passage qui y répond ne se ressemblent pas, le préfixe indique au modèle quel rôle joue chaque texte.

2.4 Index vectoriel et recherche approchée

Trouver les k vecteurs les plus proches d'une question parmi N vecteurs demande N calculs en recherche exacte. Pour de grands corpus, on utilise une **recherche approchée** (ANN). ChromaDB utilise **HNSW** (Hierarchical Navigable Small World) : un graphe à plusieurs niveaux où chaque vecteur est relié à ses voisins ; la recherche descend les niveaux en suivant les arêtes vers la question. C'est très rapide, au prix d'une petite probabilité de manquer un voisin exact.

Avec quelques centaines de chunks, ce projet n'a pas besoin d'ANN : une recherche exacte (un produit matriciel numpy) serait instantanée. ChromaDB est utilisée pour sa persistance sur disque, ses métadonnées et la facilité de passer à l'échelle, pas pour sa vitesse.

2.5 Le chunking

On n'indexe pas des documents entiers : un document de 7 000 mots dilue son sens dans un seul vecteur, et ne tiendrait pas dans le prompt. On le découpe en **chunks** (passages). La taille est un compromis :

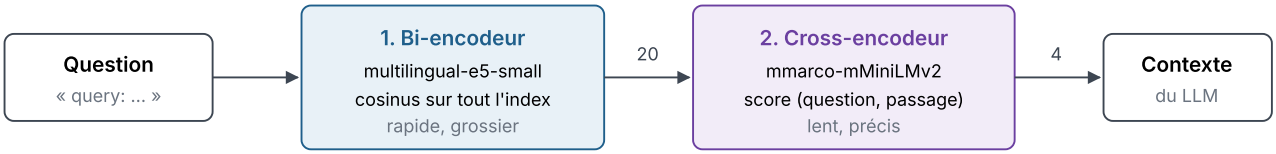
- trop petits, ils coupent les explications en deux, et le modèle reçoit des fragments sans le contexte nécessaire ;
- trop grands, ils mélangent plusieurs sujets (vecteur moins précis) et consomment la fenêtre de contexte ;
- au-delà de la longueur maximale du modèle d'embeddings (512 jetons pour E5), le texte est **tronqué silencieusement** à l'encodage : la fin du chunk n'influence pas son vecteur.

Le **chevauchement** (overlap) répète la fin d'un chunk au début du suivant, pour qu'une phrase coupée existe entière dans au moins un chunk. Le chunking peut être à taille fixe (simple, aveugle à la structure), structurel (suit les titres et paragraphes) ou sémantique (coupe là où le sens change, mesuré par embeddings). La V2 fait du chunking structurel par sections, avec un repli à taille fixe pour les sections trop longues (chapitre 5).

2.6 Bi-encodeur et cross-encodeur : le reranking

Un modèle d'embeddings est un **bi-encodeur** : la question et chaque passage sont encodés *séparément*, puis comparés. C'est ce qui permet de pré-calculer les vecteurs des passages et de chercher vite. Mais la comparaison est grossière : les deux textes ne se « voient » jamais.

Un **cross-encodeur** lit la paire (question, passage) *ensemble*, dans un seul passage du transformer, et produit un score de pertinence. Il est bien plus précis, mais il faut l'exécuter pour chaque paire : impossible sur tout le corpus. D'où la recherche en deux étapes, standard en production :



Le bi-encodeur trie tout le corpus pour ramener un rappel large ; le cross-encodeur trie finement ces 20 candidats.

Figure 1. Recherche en deux étapes (`retrieval/vector_store.py`, fonction `search`).

2.7 Prompting et ancrage

Le **prompt système** fixe les règles du modèle. Celui du projet impose : répondre uniquement à partir du contexte, refuser avec une phrase exacte si l'information manque, citer les documents par leur nom entre crochets, n'utiliser des guillemets que pour une citation mot pour mot, écrire les identifiants de code exactement comme dans le contexte. Chaque règle est **vérifiable par un programme** (chapitre 8) : c'est ce qui permet de mesurer le respect des règles sans juge et, plus tard, d'en faire une récompense pour l'entraînement.

Un prompt n'est qu'une demande : un modèle de 8B ne la respecte pas toujours. Le DPO (section 2.10) sert précisément à transformer ces règles en comportement appris.

2.8 Évaluer un système RAG

On peut évaluer à trois niveaux, du plus objectif au plus subjectif :

1. **Retrieval, contre une vérité terrain** : si l'on sait quel passage contient la réponse, on vérifie s'il a été récupéré et à quel rang. Le projet stocke pour chaque question des *preuves* recopiées du corpus ; un chunk « contient la preuve » s'il en contient le texte.
2. **Génération, par règles** : refus, sources citées, citations textuelles, identifiants de code. Déterministe, reproductible, gratuit.

3. **Génération, par un LLM juge** : le framework **Ragas** demande à un LLM de décomposer la réponse en affirmations et de vérifier chacune contre le contexte (*faithfulness*), de régénérer des questions à partir de la réponse et de mesurer leur similarité avec la question posée (*answer relevancy*), ou de juger la pertinence de chaque passage récupéré (*context precision*). Plus riche sémantiquement, mais bruité et biaisé, surtout avec un petit juge local.

Le **MRR** (Mean Reciprocal Rank) vaut 1/rang du premier résultat pertinent, moyenné sur les questions : 1 si le bon passage est premier, 0,5 s'il est deuxième, 0 s'il est absent.

Un piège classique est le **biais d'auto-évaluation** : si le juge est le même modèle que le générateur, il tend à valider ses propres formulations. La V1 utilisait le même modèle pour les deux rôles ; la V2 utilise `qwen2.5:7b` comme juge, d'une autre famille que Llama.

2.9 Statistiques : intervalle de confiance, bootstrap, test apparié

Une moyenne calculée sur 15 ou 85 questions est une **estimation** : avec d'autres questions du même type, on aurait obtenu un autre chiffre. L'**intervalle de confiance à 95 %** donne une plage plausible pour la vraie valeur. Pour une métrique binaire (0 ou 1) de moyenne p sur n questions, l'écart-type de la moyenne vaut environ $\sqrt{p(1-p)/n}$: avec p = 0,8 et n = 15, cela fait 0,10, donc un intervalle d'environ ±0,20. Un écart de 0,07 entre deux configurations évaluées sur 15 questions ne dit donc rien.

Le **bootstrap** estime cet intervalle sans hypothèse sur la distribution : on tire avec remise n questions parmi les n, on recalcule la moyenne, on répète 2 000 fois, et on prend les quantiles 2,5 % et 97,5 % des moyennes obtenues (méthode des percentiles).

Pour **comparer deux versions**, il faut les évaluer sur les mêmes questions et raisonner sur les *différences question par question* : c'est le **test apparié**. Les questions difficiles le sont pour les deux versions ; travailler sur les différences élimine cette variabilité commune et rend le test beaucoup plus sensible qu'une comparaison de deux moyennes. Le projet applique un bootstrap sur les différences : si l'intervalle de la différence exclut 0, l'écart est significatif.

Il existe une seconde source de variance : à juge et générateur fixés, relancer l'évaluation ne donne pas exactement les mêmes scores (échantillonnage du juge, non-déterminisme du GPU). L'option `--repeats` mesure cette variance entre exécutions.

2.10 Alignement par préférences : RLHF, PPO, DPO

Après le pré-entraînement et le fine-tuning supervisé, on **aligne** un modèle sur des préférences : préférer la réponse A à la réponse B pour un même prompt. L'approche historique, le **RLHF** (Reinforcement Learning from Human Feedback), entraîne d'abord un *modèle de récompense* sur des paires annotées, puis optimise le LLM contre cette récompense avec **PPO**, un algorithme d'apprentissage par renforcement, en le pénalisant (divergence KL) s'il s'éloigne trop du modèle d'origine. C'est puissant mais lourd : quatre modèles en mémoire (politique, référence, récompense, critique), réglage délicat, instabilité.

DPO (Direct Preference Optimization, Rafailov et al., 2023) montre qu'on peut obtenir la même solution optimale sans modèle de récompense ni boucle de renforcement, avec une simple perte de classification sur les paires (y_w préférée, y_l rejetée) :

$$L = -\log \sigma(\ \beta\ [\ (\log \pi_\theta(y_w|x) - \log \pi_{ref}(y_w|x)) - (\log \pi_\theta(y_l|x) - \log \pi_{ref}(y_l|x))\ ]\ )$$

π_θ est le modèle entraîné, π_{ref} le modèle de référence (gelé), σ la sigmoïde, β règle l'écart autorisé par rapport à la référence. Intuitivement, la perte pousse le modèle à augmenter la probabilité relative de la réponse préférée et à diminuer celle de la réponse rejetée, *par rapport à ce que faisait la référence*. La quantité $\beta(\log \pi_\theta - \log \pi_{ref})$ s'interprète comme une « récompense implicite » ; TRL journalise son exactitude (part des paires où la préférée l'emporte) et sa marge.

Le DPO reste une méthode d'apprentissage par préférences : la qualité du résultat dépend entièrement de la qualité des paires. Si la réponse « préférée » n'est pas réellement meilleure, le modèle apprend le mauvais comportement. C'est pourquoi le projet construit les paires à partir d'une **récompense vérifiable** plutôt que d'un juge LLM (chapitre 11).

2.11 LoRA et QLoRA

Ré-entraîner les 8 milliards de paramètres d'un modèle demande des dizaines de Go de mémoire GPU. **LoRA** (Low-Rank Adaptation, Hu et al., 2021) gèle les poids d'origine W_0 et n'apprend qu'une correction de faible rang :

$$W = W_0 + (\alpha / r) \cdot B\ A, \quad \text{avec } A \in \mathbb{R}^{r \times d},\ B \in \mathbb{R}^{d \times r},\ r \ll d$$

Avec r = 16 sur des matrices de dimension 4 096, on entraîne une fraction de pour cent des paramètres. L'adaptateur obtenu (quelques dizaines de Mo) s'applique au modèle de base ou s'y fusionne.

QLoRA (Dettmers et al., 2023) charge en plus le modèle de base en 4 bits (format NF4, adapté à la distribution des poids) et calcule en 16 bits. Un modèle de 8B tient alors dans environ 6 Go, ce qui rend l'entraînement possible sur un GPU T4 de 16 Go (Colab gratuit). Avantage supplémentaire avec DPO : le modèle de référence est simplement le même modèle *adaptateurs désactivés*, sans seconde copie en mémoire.

2.12 NLI : vérifier qu'une phrase découle d'un texte

L'**inférence en langage naturel** (NLI) classe une paire (prémisse, hypothèse) en *implication*, *contradiction* ou *neutre*. Un modèle NLI répond donc à la question « cette phrase de la réponse découle-t-elle de ce passage ? ». C'est une version spécialisée, légère et déterministe de ce que fait la faithfulness de Ragas avec un LLM. Le projet utilise un modèle NLI multilingue (mDeBERTa-v3, entraîné sur XNLI et d'autres jeux) dans la récompense DPO : les phrases de la réponse sont en français, les passages en anglais.

PARTIE II

Architecture

La vue d'ensemble : les quatre chaînes du système, les artefacts qu'elles produisent et consomment, l'organisation du dépôt, la stack et la configuration.

3 Vue d'ensemble

Le système est un ensemble de scripts Python indépendants qui communiquent par des fichiers (corpus, chunks, index, résultats, paires de préférences, adaptateur). Ce choix rend chaque étape exécutable, testable et inspectable isolément.

3.1 Les quatre chaînes

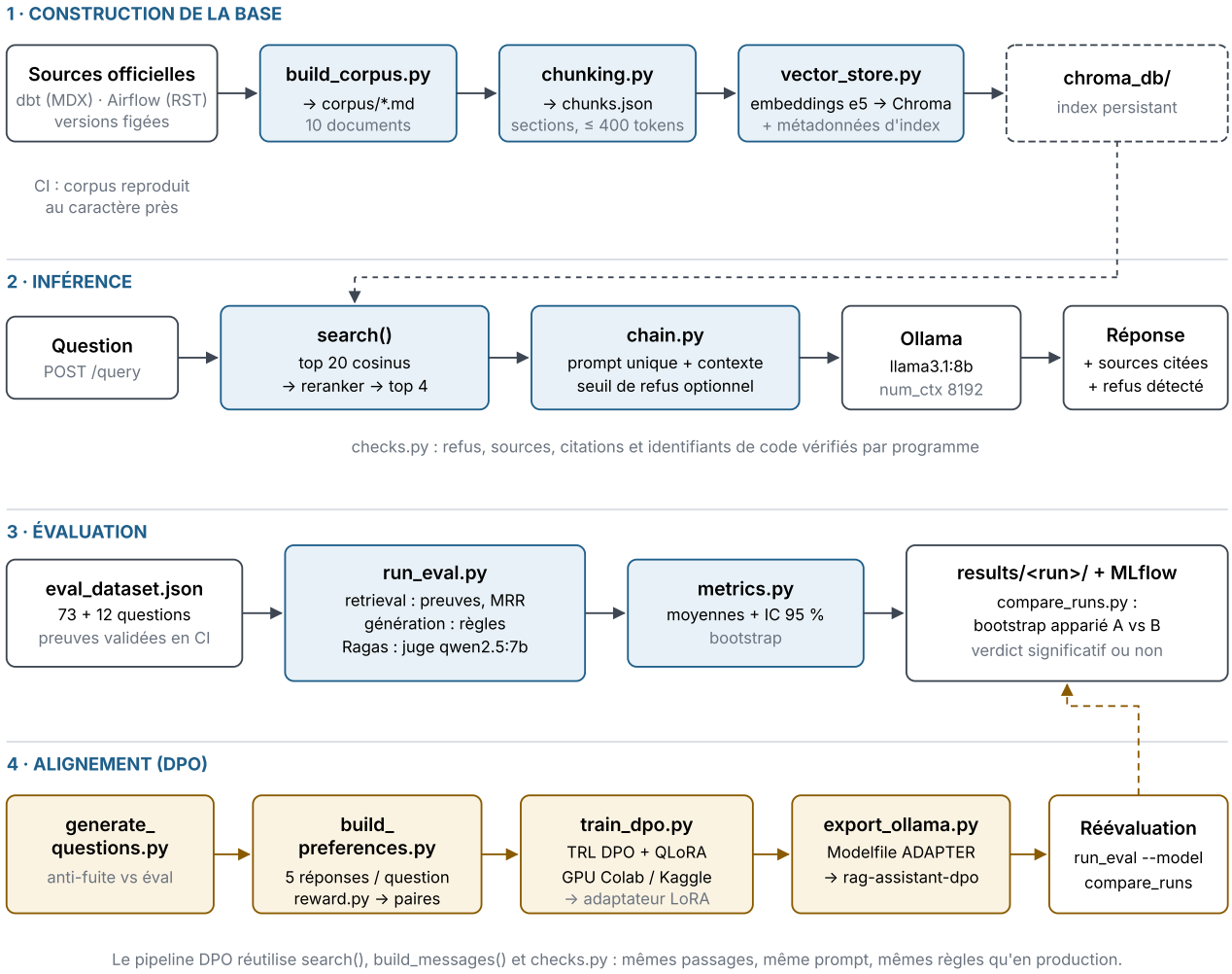


Figure 2. Les quatre chaînes du système et les fichiers qui les relient. En bleu : modules exécutés en local ; en ocre : pipeline d'alignement.

Un principe traverse le projet : **une seule définition de chaque chose**. Le prompt est défini une fois (`build_messages`) et utilisé par l'inférence, l'évaluation et l'entraînement. Les règles de vérification sont définies une fois (`checks.py`) et servent à la fois de métriques d'évaluation et de récompense DPO. La recherche est définie une fois (`search`) et alimente l'API, l'évaluation et la génération des paires. Si ces trois usages divergeaient, on entraînerait le modèle sur une situation différente de celle qu'on évalue et de celle qu'on sert.

3.2 Artefacts et flux de données

Artefact	Produit par	Consommé par	Versionné ?
`corpus/*.md`	`build_corpus`	`chunking`, `dataset`	Oui ; la CI vérifie qu'il se reconstruit à l'identique.
`ingestion/chunks.json`	`chunking`	`vector_store`, `generate_questions`	Non (dérivé) ; construit dans l'image Docker.
`chroma_db/`	`vector_store`, API	`search()`	Non ; volume Docker.
`evaluation/eval_dataset.json`	Rédigé à la main	`run_eval`, anti-fuite	Oui ; preuves validées en CI.
`evaluation/results/<run>/`	`run_eval`	`compare_runs`	Au choix (runs de référence).
`mlruns.db`	`mlflow_tracking`	`mlflow ui`	Non.
`training/data/questions.jsonl`	`generate_questions`	`build_preferences`	Non.
`training/data/preferences.jsonl`	`build_preferences`	`train_dpo` (Colab)	Non.
`training/outputs/dpo-lora/`	`train_dpo`	`export_ollama`	Non.

3.3 Organisation du dépôt

```
assistant-rag/
├── config.py                    configuration centrale (variables d'environnement)
├── corpus/                      10 documents Markdown reconstruits depuis les sources
├── ingestion/
│   ├── build_corpus.py          MDX / reStructuredText → Markdown (versions figées)
│   ├── chunking.py              découpage par sections, fil d'Ariane
│   └── embeddings.py            embeddings multilingues + reranker
├── retrieval/vector_store.py    ChromaDB, reranking, métadonnées et fraîcheur de l'index
├── rag_chain/
│   ├── chain.py                 prompt unique, génération, seuil de refus
│   └── checks.py                vérifications déterministes des réponses
├── api/main.py                  FastAPI : POST /query, GET /health
├── evaluation/
│   ├── eval_dataset.json        85 questions avec preuves
│   ├── dataset.py               chargement et validation contre le corpus
│   ├── metrics.py               métriques, bootstrap, seuil
│   ├── run_eval.py              boucle d'évaluation + MLflow
│   ├── compare_runs.py          comparaison appariée de deux runs
│   ├── mlflow_tracking.py       journalisation MLflow
│   ├── _ragas_compat.py         correctif d'import ragas / langchain-community
│   └── results/v1/              résultats archivés de la V1
├── training/
│   ├── reward.py                récompense vérifiable
│   ├── generate_questions.py    questions d'entraînement + anti-fuite
│   ├── out_of_scope_train.json  60 questions hors périmètre d'entraînement
│   ├── build_preferences.py     paires chosen / rejected
│   ├── train_dpo.py             DPO + QLoRA (TRL)
│   ├── dpo_colab.ipynb          notebook d'entraînement GPU
│   ├── export_ollama.py         adaptateur ou modèle fusionné → Ollama
│   └── README.md                procédure pas à pas
├── tests/                       52 tests (sans modèle ni GPU)
├── .github/workflows/           ci-cd.yml, training-smoke.yml
├── Dockerfile, docker-compose.yml, .dockerignore
```

```
├── pyproject.toml                configuration ruff et pytest
└── requirements.txt, requirements-dev.txt, requirements-train.txt
```

Environ 3 600 lignes de Python (tests compris) et 30 000 mots de corpus.

3.4 Stack technique

Brique	Outil	Rôle et raison du choix
Générateur	Ollama + `llama3.1:8b` (Q4_K_M)	Gratuit, local, tient sur un GPU de 8 Go. Ollama gère la quantification, le gabarit de chat et une API HTTP.
Juge d'évaluation	Ollama + `qwen2.5:7b`	Famille différente du générateur, pour limiter le biais d'auto-évaluation.
Embeddings	`intfloat/multilingual-e5-small`	Multilingue (questions FR, corpus EN), petit (384 dimensions), tourne sur CPU.
Reranker	`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`	Cross-encodeur multilingue entraîné sur mMARCO (MS MARCO traduit), léger.
Base vectorielle	ChromaDB 1.5 (client persistant)	Sans serveur, fichiers locaux, métadonnées par collection.
Orchestration	LangChain (LCEL), `langchain-ollama`	Composition prompt → LLM → parseur ; découpeur de texte récursif.
API	FastAPI + Uvicorn, Pydantic	Validation des entrées, documentation Swagger automatique, cycle de vie (indexation au démarrage).
Évaluation	Ragas 0.4.3, numpy, pandas	Métriques sémantiques standard ; le reste est calculé par le projet.
Suivi	MLflow 3 (backend SQLite)	Historique des runs, paramètres, métriques, artefacts.
Alignement	TRL 1.15, PEFT 0.21, bitsandbytes, transformers 5.19	Implémentation de référence de DPO, LoRA/QLoRA.
Récompense NLI	`MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7`	NLI multilingue pour le score de grounding.
Qualité	pytest, ruff	Tests sans modèle, lint et tri des imports.
Déploiement	Docker, GitHub Actions, GHCR	Image de l'API, CI/CD, registre d'images gratuit lié au dépôt.

3.5 Configuration centrale

`config.py` lit toutes les valeurs depuis les variables d'environnement (fichier `.env` chargé par `python-dotenv`), avec des valeurs par défaut. Aucun autre module ne lit l'environnement directement : changer de modèle ou de taille de chunk se fait à un seul endroit, sans toucher au code. La fonction interne `_optional_float` transforme une variable vide en `None` (utilisée pour le seuil de refus, désactivé par défaut).

Variable	Défaut	Rôle
`OLLAMA_BASE_URL`	`http://127.0.0.1:11434`	Serveur Ollama (port modifiable si réservé par Windows).

Variable	Défaut	Rôle
`OLLAMA_LLM_MODEL`	`llama3.1:8b`	Générateur. Devient `rag-assistant-dpo` si le DPO est adopté.
`OLLAMA_NUM_CTX`	`8192`	Fenêtre de contexte demandée à Ollama.
`JUDGE_LLM_MODEL`	`qwen2.5:7b`	Juge Ragas, rédacteur des questions d'entraînement.
`EMBEDDING_MODEL_NAME`	`intfloat/multilingual-e5-small`	Modèle d'embeddings.
`EMBEDDING_QUERY_PREFIX` / `_PASSAGE_PREFIX`	`"query: "` / `"passage: "` si E5, sinon vides	Préfixes asymétriques.
`CHUNK_SIZE_TOKENS`	`400`	Taille maximale d'un chunk (tokens du tokenizer d'embeddings).
`CHUNK_OVERLAP_TOKENS`	`50`	Chevauchement lors du recoupage d'une longue section.
`CHROMA_PERSIST_DIR`	`./chroma_db`	Dossier de l'index.
`CHROMA_COLLECTION_NAME`	`dbt_airflow_docs`	Nom de la collection.
`TOP_K`	`4`	Passages transmis au LLM.
`RETRIEVAL_CANDIDATES`	`20`	Candidats de la recherche vectorielle avant reranking.
`RERANKER_MODEL_NAME`	mmarco-mMiniLMv2 (vide = désactivé)	Cross-encodeur.
`RELEVANCE_THRESHOLD`	vide (désactivé)	Refus sans appel au LLM si le meilleur score est inférieur.
`MLFLOW_TRACKING_URI`	`sqlite:///mlruns.db`	Backend MLflow.
`MLFLOW_EXPERIMENT_NAME`	`rag-genai-eval`	Expérience MLflow.

Constantes de chemins (non surchargeables) : `ROOT_DIR`, `CORPUS_DIR`, `CHUNKS_PATH`, `TRAINING_DIR`, `TRAINING_DATA_DIR`. Côté Docker, `OLLAMA_BASE_URL_DOCKER` (défaut `http://host.docker.internal:11434`) et `HOST_PORT` (défaut 8600) sont lus par `docker-compose.yml`.

> **CE QU'ON AURAIT PU FAIRE**
>
> - **pydantic-settings** : une classe `Settings` typée et validée (erreur claire si `TOP_K=abc`). Plus robuste, au prix d'un objet à passer partout.
> - **Fichier YAML de configuration + Hydra** : pratique pour lancer des grilles d'expériences (taille de chunk × modèle × top-k). Surdimensionné pour ce projet.
> - **Packaging Python** (un paquet installable avec `pip install -e .`) : supprimerait les `sys.path.insert` en tête de chaque script, un raccourci volontaire pour garder des scripts lançables directement.

PARTIE III

Les modules en détail

Chaque module est présenté de la même façon : son rôle, son fonctionnement, la liste de ses fonctions, les choix faits et pourquoi, puis ce qu'on aurait pu faire à la place.

4 Construction du corpus

`ingestion/build_corpus.py` (484 lignes) transforme les sources de 10 pages de documentation officielle en 10 fichiers Markdown propres, reproductibles au caractère près.

4.1 Rôle et sources

Le corpus couvre un domaine cohérent, l'outillage data engineering :

Fichier	Page	Mots
`dbt_01_sql_models.md`	Modèles SQL, configuration, `ref`, FAQ	2 739
`dbt_02_jinja_macros.md`	Jinja, macros, packages, bonnes pratiques	1 819
`dbt_03_tests.md`	Data tests singuliers et génériques, stockage des échecs	3 189
`dbt_04_incremental_models.md`	Modèles incrémentaux, `unique_key`, `on_schema_change`	2 316
`dbt_05_materializations.md`	view, table, incremental, ephemeral, materialized view	1 433
`airflow_01_dags.md`	Déclaration, dépendances, branchements, trigger rules	5 749
`airflow_02_operators.md`	Opérateurs, templating Jinja	1 809
`airflow_03_scheduler.md`	Scheduler, haute disponibilité, réglages	2 129
`airflow_04_xcoms.md`	XComs, backends	794
`airflow_05_best_practices.md`	Bonnes pratiques d'écriture et de test des Dags	7 701

Les sources sont lues dans les dépôts GitHub officiels, à des versions **figées dans le code** : le commit `3aea3292…` de `dbt-labs/docs.getdbt.com` (fichiers MDX) et le tag `3.3.1` d'`apache/airflow` (fichiers reStructuredText). Chaque document commence par son titre, préfixé du produit (« dbt — », « Apache Airflow — »), et une ligne `Source :` qui indique l'URL publique et la version.

4.2 Fonctionnement

Côté dbt : convertir du MDX

Le site de dbt est écrit en MDX : du Markdown mêlé de composants React. Le convertisseur `MdxConverter` :

- lit le *front matter* YAML (titre) ;
- sépare le texte en segments « prose » et « bloc de code » (`_split_fences`), pour ne jamais modifier le code ;
- parcourt les balises JSX avec un petit analyseur (`_scan_jsx_tags`) qui gère les balises multilignes et les props entre accolades ;
- remplace chaque composant par son équivalent textuel : `<Constant name="core"/>` par sa valeur lue dans `constants.js` du site (« dbt v1 ») ; `<Term id="data-warehouse"/>` par « data warehouse » ; `<File name="…">` par « File `…`: » ; `<TabItem>` par le libellé de l'onglet ; `<FAQ path="…"/>` par le contenu de la FAQ, téléchargé et inséré comme sous-section ; un fragment importé depuis `/snippets/` par son contenu, avec substitution des `{props.x}` ;

- convertit les encadrés `:::info` en « Info: … », supprime les images, garde le texte des liens.

Côté Airflow : convertir du reStructuredText

`RstConverter` parcourt le fichier ligne par ligne :

- **titres** : en RST, un titre est une ligne soulignée (`=====`, `-----` …), et le niveau dépend de l'ordre d'apparition des styles de soulignement. Le convertisseur mémorise cet ordre pour produire `#`, `##`, `###` ;
- **directives** (`.. code-block::`, `.. note::`, `.. versionadded::` …) : converties en blocs de code ou en paragraphes « Note: … » ; les images et tables des matières sont supprimées ;
- **inclusions** : `.. include::` et `.. exampleinclude::` (exemples de code tirés des Dags d'exemple d'Airflow) sont résolues en téléchargeant le fichier cible et en extrayant la portion entre les marqueurs `:start-after:` et `:end-before:` ;
- **rôles en ligne** (`rst_inline`) : `:class:`~airflow.sdk.BaseXCom`` devient `` `BaseXCom` ``, `:ref:`texte <cible>`` devient « texte », ``` ``code`` ``` devient `` `code` `` ;
- **blocs littéraux** introduits par `::` en fin de paragraphe : convertis en bloc de code ;
- **commentaires et cibles** (en-tête de licence, `.. _label:`) : supprimés.

Enfin, `build_document` supprime les sections de pure navigation (« Related reference docs », listes de liens), ajoute l'en-tête et normalise les espaces (`_tidy`).

4.3 Fonctions

Fonction	Rôle
`http_fetch` `(url) → str`	Télécharge une URL (requests, timeout 30 s). Injectable : les tests passent un faux `fetch`.
`_split_fences` `(text) → [(est_code, segment)]`	Sépare prose et blocs de code délimités par ``` ``` ``` ou `~~~`.
`_drop_sections` `(markdown, titles) → str`	Supprime un titre de navigation et la liste de liens qui le suit ; s'arrête au premier paragraphe non-liste.
`_tidy` `(text) → str`	Supprime les espaces de fin de ligne et les lignes vides multiples.
`parse_frontmatter` `(text) → (meta, corps)`	Extrait le bloc YAML entre `---` (clés simples).
`_scan_jsx_tags` `(text) → itérateur`	Repère les balises JSX ouvrantes, fermantes ou auto-fermantes, en suivant guillemets et accolades.
`_prop`, **`_term_label`**	Lit un attribut de balise ; transforme un identifiant de terme en libellé (« cte » → « CTE »).
`MdxConverter` `(fetch, raw_base)`	Convertisseur MDX. Méthodes : `constants` (lit `constants.js`, en cache), `convert(text, heading_shift)` → (titre, markdown), `_clean_code_fence`, `_convert_prose`, `_faq`.
`rst_inline` `(text) → str`	Convertit les rôles et liens RST en ligne.

Fonction	Rôle
`RstConverter` `(fetch, raw_base, docs_root)`	Convertisseur RST. Méthodes : `convert`, `_resolve` (chemins Sphinx relatifs), `_slice` (marqueurs start/end), `_convert_lines` (parcours principal), `_directive`.
`build_document` `(name, public_url, source_url, fetch) → str`	Produit un document complet : conversion, suppression de la navigation, en-tête, nettoyage.
`main` `()`	Construit les 10 documents de `PAGES` dans `corpus/`.

4.4 Choix et alternatives

POURQUOI PARTIR DES SOURCES PLUTÔT QUE DU SITE

La V1 téléchargeait le HTML rendu des sites, en extrayait le texte avec BeautifulSoup, puis générait des PDF avec ReportLab, que le chunking relisait avec pypdf. Trois défauts : chaque élément en ligne (lien, `code`) devenait une ligne à part et coupait les phrases ; des artefacts d'interface (« ↑↓ Navigate ■ Select Esc Close ») se retrouvaient dans le texte ; la structure en sections était perdue, ce qui interdisait un chunking structurel. Le passage par PDF n'apportait rien : on écrivait un format de mise en page pour le relire aussitôt. Les sources donnent un texte propre, structuré et versionné ; figer les versions rend le corpus reproductible, ce que la CI vérifie à chaque push en le reconstruisant et en exigeant un `git diff` vide.

CE QU'ON AURAIT PU FAIRE

- **Construire les sites avec leurs propres outils** (Docusaurus pour dbt, Sphinx pour Airflow) et convertir le HTML produit : fidèle à 100 %, mais il faut installer deux chaînes de build complètes. Les convertisseurs maison couvrent les constructions effectivement présentes dans ces 10 pages, et les tests vérifient chacune d'elles.
- **Pandoc** pour le RST : convertisseur générique et robuste, mais il ne connaît pas les directives propres à Sphinx et Airflow (`exampleinclude`, rôles `:class:`) et ne résout pas les inclusions.
- **Unstructured, Docling ou un parseur de PDF avancé** : utiles quand on n'a que des PDF ou des documents bureautiques. Ici, on a mieux : la source.
- **Garder le HTML et extraire avec trafilatura** : plus simple, mais dépend de la mise en page du site, qui change sans prévenir.

ATTENTION

Les convertisseurs sont spécialisés : une page qui utiliserait un composant MDX ou une directive RST non prévus laisserait passer des balises brutes. Ajouter une page au corpus demande de relire le Markdown produit.

5 Chunking

`ingestion/chunking.py` (261 lignes) découpe les 10 documents en passages par sections, chacun préfixé de son fil d'Ariane, et enregistre les paramètres qui les ont produits.

5.1 Fonctionnement

L'algorithme (`CHUNKER_VERSION = "sections-v2"`) se déroule en trois temps :

1. **Découpage en sections** (`parse_sections`) : chaque titre Markdown ouvre une section ; le chemin de titres (« Apache Airflow — XComs > Custom XCom Backends ») est conservé. Les `#` à l'intérieur des blocs de code (commentaires Python) sont ignorés. La ligne `Source :` est extraite comme URL.
2. **Regroupement** : une petite section est fusionnée avec la précédente si elles partagent la même section de niveau 2 et si le total tient dans la taille maximale. Le titre de la sous-section fusionnée reste dans le texte. Cela évite des chunks de deux lignes, trop pauvres pour un bon vecteur.
3. **Recoupage** : une section plus longue que le budget est coupée par `RecursiveCharacterTextSplitter` (LangChain), qui essaie dans l'ordre les séparateurs paragraphe, bloc de code, ligne, phrase, mot ; avec un chevauchement de 50 tokens.

Chaque chunk commence par son fil d'Ariane, suivi d'une ligne vide :

```
Apache Airflow — XComs > Custom XCom Backends

The XCom system has interchangeable backends, and you can set which backend is
being used via the `xcom_backend` configuration option. …
```

Le fil d'Ariane rend le chunk compréhensible hors de son document (pour le LLM comme pour le lecteur) et injecte dans le vecteur le sujet de la section, souvent absent du texte lui-même.

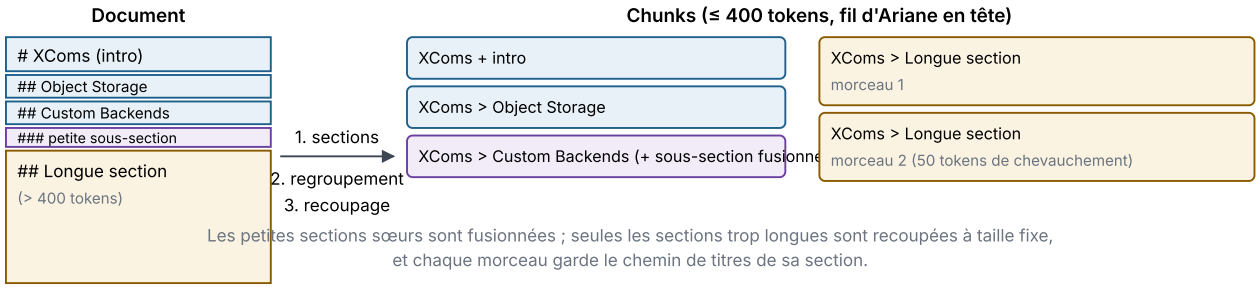


Figure 3. Chunking par sections (`chunk_document`).

Mesurer la taille avec le bon tokenizer

La taille est mesurée avec le **tokenizer du modèle d'embeddings** (`get_token_counter`), pas avec celui d'OpenAI (`tiktoken cl100k_base`) comme en V1. Ce qui compte est ce que le modèle d'embeddings peut encoder : 512 tokens au maximum pour E5, au-delà le texte est tronqué. 400 tokens laissent de la place au préfixe `"passage: "` et au fil d'Ariane. Au moment de l'indexation, `count_truncated` compte les chunks qui dépasseraient malgré tout et affiche un avertissement.

Traçabilité

`chunks.json` contient, en plus des chunks, les paramètres qui les ont produits (`chunking_params`) : empreinte SHA-256 du corpus (12 premiers caractères), version de l'algorithme, taille, chevauchement, tokenizer. Ces paramètres sont recopiés dans les métadonnées de l'index, puis dans MLflow. Chaque chunk porte un identifiant stable (`airflow_04_xcoms.md::chunk_003`), son document, son rang, sa section, l'URL source, son texte et sa taille en tokens.

5.2 Fonctions

Fonction	Rôle
`Chunk`, **`Section`**	Structures de données (dataclasses). `Section.body` renvoie le texte de la section.
`get_token_counter` `() → (str → int)`	Charge le tokenizer du modèle d'embeddings (sans le modèle), une seule fois.
`load_pdf_text` `(pdf_path) → str`	Extraction de texte PDF (pypdf), conservée pour pouvoir ajouter des PDF au corpus.
`load_corpus_documents` `(corpus_dir) → [{source, text}]`	Charge les `.md` (et `.pdf`) du corpus ; ignore les textes vides (PDF scannés).
`parse_sections` `(markdown) → (url, [Section])`	Découpe en sections avec chemins de titres, en ignorant le code.
`chunk_document` `(source, text, count_tokens, chunk_size, chunk_overlap) → [Chunk]`	Regroupement puis recoupage d'un document.
`chunk_documents` `(documents, count_tokens=None, …)`	Applique `chunk_document` à tout le corpus.
`corpus_fingerprint` `(documents) → str`	Empreinte du corpus, pour détecter un index obsolète.
`chunking_params`, **`save_chunks`**, **`load_chunks`**	Paramètres de production ; écriture et lecture de `chunks.json` (l'ancien format en liste nue reste lisible).

5.3 Choix et alternatives

> **POURQUOI UN CHUNKING PAR SECTIONS**
>
> La documentation technique est déjà découpée par ses auteurs en sections cohérentes : c'est le meilleur découpage disponible, et il ne coûte rien. L'itération de la V1 (300 tokens au lieu de 500) avait fait baisser la faithfulness, un indice que la coupe à taille fixe tronquait les explications, même si cette mesure était trop bruitée pour conclure (chapitre 14).

CE QU'ON AURAIT PU FAIRE

- **Chunking sémantique** : couper là où la similarité entre phrases consécutives chute. Utile pour du texte sans structure (transcriptions, e-mails), inutile quand les titres existent.
- **Parent-child (small-to-big)** : indexer de petits chunks pour une recherche précise, mais envoyer au LLM la section parente entière. Bon compromis précision/contexte ; c'est l'évolution la plus naturelle si les réponses manquent de contexte.
- **Contextual retrieval** : faire écrire par un LLM, pour chaque chunk, une phrase qui le situe dans son document, et l'ajouter avant l'indexation. Le fil d'Ariane en est une version gratuite et déterministe.
- **Late chunking** : encoder le document entier avec un modèle à long contexte, puis découper les vecteurs de tokens. Demande un modèle d'embeddings à long contexte.
- **Propositions** : faire réécrire le texte en affirmations atomiques. Coûteux et risque de déformer la source.

ATTENTION

Mesuré sur le corpus réel avec le tokenizer E5 : **207 chunks**, médiane de 311 tokens, maximum de 400, aucun chunk tronqué à l'encodage ; 202 font au moins 60 tokens et servent à générer les questions d'entraînement. Les tests utilisent un compteur approximatif injecté : ils vérifient la logique, pas la taille exacte en tokens E5.

6 Embeddings et reranker

`ingestion/embeddings.py` (80 lignes) charge les deux modèles de sentence-transformers et expose des fonctions simples d'encodage et de scoring.

6.1 Fonctions

Fonction	Rôle
`get_embedding_model` `()`	Charge `SentenceTransformer(EMBEDDING_MODEL_NAME)` une seule fois (`lru_cache`).
`_encode` `(texts) → vecteurs`	Encode avec normalisation L2 ; barre de progression au-delà de 20 textes.
`embed_passages` `(texts)`	Encode des chunks avec le préfixe `"passage: "`.
`embed_queries`, **`embed_query`**	Encode des questions avec le préfixe `"query: "`.
`count_truncated` `(texts) → int`	Nombre de passages plus longs que `max_seq_length` du modèle.
`get_reranker` `()`	Charge le `CrossEncoder` (longueur max 512), ou `None` si désactivé.
`rerank_scores` `(query, passages) → [float] | None`	Score de pertinence de chaque paire (question, passage).

Les imports de `sentence_transformers` sont faits à l'intérieur des fonctions : le module s'importe sans PyTorch, ce qui permet aux tests et à la CI de tourner sans installer les modèles.

6.2 Choix et alternatives

> **POURQUOI CES DEUX MODÈLES**
>
> Ils sont multilingues, petits (ils tournent sur CPU, y compris dans le conteneur Docker), gratuits et locaux. `multilingual-e5-small` est un bon compromis taille/qualité parmi les modèles multilingues ouverts ; le cross-encodeur mMARCO est entraîné précisément sur une tâche question → passage, dans plusieurs langues.

CE QU'ON AURAIT PU FAIRE

- **Modèles plus grands** : `multilingual-e5-base` ou `-large`, `BAAI/bge-m3` (multilingue, 8 192 tokens de contexte, vecteurs denses et creux). Meilleurs, mais plus lents sur CPU. À tester avec `run_eval.py --retrieval-only`, qui mesure l'effet en quelques secondes.
- **Reranker plus fort** : `BAAI/bge-reranker-v2-m3`, multilingue et plus précis, environ 570 M de paramètres.
- **API commerciales** (OpenAI, Cohere, Voyage) : excellentes, mais payantes et non locales, contraires au cahier des charges.
- **Traduire les questions en anglais** avant la recherche : contourne le problème cross-lingue avec un modèle anglais, au prix d'un appel LLM et d'erreurs de traduction sur les termes techniques.
- **Fine-tuner le modèle d'embeddings** sur des paires (question, chunk) générées : gain possible sur le domaine, mais il faut beaucoup de paires et on risque le surapprentissage sur un corpus de 10 documents.

7 Base vectorielle et recherche

`retrieval/vector_store.py` (205 lignes) indexe les chunks dans ChromaDB, vérifie que l'index correspond à la configuration courante, et implémente la recherche en deux étapes.

7.1 Fonctionnement

Indexation

`index_chunks` lit `chunks.json`, encode les textes, supprime puis recrée la collection, et y enregistre les vecteurs, les textes et les métadonnées de chaque chunk (document, rang, section, URL). Les métadonnées **de la collection** reçoivent la configuration d'indexation, préfixée par `index_` : modèle d'embeddings, préfixes, nombre de chunks, nombre de chunks tronqués, date, empreinte du corpus et paramètres de chunking.

Contrôles

- `ensure_index_ready`, appelée à chaque recherche, lève `IndexNotReadyError` si l'index est vide, s'il a été construit par une ancienne version (pas de métadonnées), ou avec un autre modèle d'embeddings que celui configuré. Sans ce contrôle, changer de modèle sans réindexer donnerait des résultats absurdes *sans aucune erreur* : les vecteurs de deux modèles différents ne sont pas comparables.
- `index_is_stale` compare l'index à `chunks.json` (empreinte du corpus, nombre de chunks, paramètres). L'API s'en sert au démarrage pour réindexer automatiquement.

Recherche

`search(query, k=4, candidates=20, rerank=True)` récupère 20 candidats par similarité cosinus, les fait noter par le cross-encodeur, les retrie et renvoie les 4 meilleurs. Chaque résultat contient `id`, `text`, `source`, `section`, `url`, `similarity` (cosinus), `rerank_score` et `score` (le critère de tri : score du reranker s'il est actif, cosinus sinon). Ce `score` sert aussi au seuil de refus optionnel.

7.2 Fonctions

Fonction	Rôle
`IndexNotReadyError`	Exception : index absent ou incompatible. L'API la traduit en HTTP 503.
`get_client`, **`get_collection`**	Client persistant ChromaDB (créé une fois, import paresseux) ; collection en espace cosinus.
`get_index_metadata` `() → dict`	Configuration enregistrée lors de l'indexation.
`index_chunks` `(chunks_path) → int`	Reconstruit l'index de zéro.
`ensure_index_ready` `()`	Vérifie présence, métadonnées et modèle d'embeddings.
`index_is_stale` `(chunks_path) → bool`	Index vide ou différent de `chunks.json` / du modèle.

Fonction	Rôle
`search` `(query, k, candidates, rerank) → [hit]`	Recherche vectorielle puis reranking. Alias `similarity_search` pour la compatibilité V1.
`main` `()`	Indexe et affiche les résultats de 4 questions témoins, dont une hors corpus.

7.3 Choix et alternatives

CE QU'ON AURAIT PU FAIRE

- **Recherche hybride BM25 + vecteurs** : BM25 (recherche lexicale) excelle sur les identifiants exacts (`on_schema_change`, `xcom_pull`) que les embeddings rapprochent mal ; on fusionne les deux classements (Reciprocal Rank Fusion). Souvent le meilleur gain pour de la documentation technique. Limite ici : BM25 ne fait pas le pont français → anglais, sauf sur les identifiants, mais c'est justement là qu'il est utile.
- **Autres bases** : Qdrant (serveur, filtres riches, recherche hybride native), pgvector (si une base Postgres existe déjà), FAISS (bibliothèque en mémoire, très rapide, sans persistance de métadonnées), LanceDB (fichiers, sans serveur). À cette échelle, le choix n'a aucun effet sur la qualité ; c'est une question d'exploitation.
- **Réécriture de requête** : HyDE (faire générer une réponse hypothétique et chercher avec elle), multi-query (plusieurs reformulations), décomposition des questions composées. Ajoute un appel LLM par question.
- **MMR** (Maximal Marginal Relevance) : diversifier les 4 passages pour éviter quatre chunks redondants.
- **GraphRAG** : extraire un graphe d'entités et de relations (Dag → tâche → opérateur) et y naviguer. Pertinent pour des questions transverses (« quels mécanismes permettent à deux tâches de communiquer ? »), lourd à construire et à évaluer.

8 Chaîne RAG et vérifications

`rag_chain/chain.py` (132 lignes) assemble le contexte, le prompt et l'appel au LLM. `rag_chain/checks.py` (152 lignes) vérifie mécaniquement qu'une réponse respecte les règles du prompt.

8.1 Le prompt

```
Tu es un assistant technique qui répond à des questions sur la documentation dbt
et Apache Airflow.

Règles strictes :
- Réponds UNIQUEMENT à partir du contexte fourni ci-dessous. N'utilise aucune
  autre connaissance.
- Si le contexte ne contient pas la réponse, réponds exactement : "Je ne trouve
  pas cette information dans les documents fournis." et rien d'autre. N'invente pas.
- Cite les documents qui justifient ta réponse par leur nom entre crochets, tel
  qu'il apparaît dans le contexte, par exemple [airflow_04_xcoms.md]. Ne cite
  jamais un document absent du contexte.
- N'utilise des guillemets que pour recopier un passage du contexte mot pour mot,
  dans sa langue d'origine (anglais). Sinon, reformule sans guillemets.
- Écris les noms de fonctions, de paramètres et de commandes exactement comme
  dans le contexte, entre accents graves (par exemple `xcom_push`).
- Réponds en français, de façon concise et précise.

Contexte :
[Document : airflow_04_xcoms.md | Section : Apache Airflow — XComs]
…
---
[Document : …]
```

Chaque règle a été écrite pour être vérifiable : une phrase de refus exacte, des citations dans un format reconnaissable, des guillemets réservés au verbatim, des identifiants entre accents graves. La règle sur les guillemets répond à un défaut observé en V1 : le modèle présentait entre guillemets des passages reformulés.

8.2 Fonctions de chain.py

Fonction	Rôle
`format_context` `(hits) → str`	Met en forme les passages avec document et section, séparés par `---`.
`build_messages` `(question, hits) → [{role, content}]`	Messages exactement tels que le LLM les reçoit. **Seule définition du prompt**, réutilisée pour construire les exemples DPO.
`get_llm` `(model, temperature, seed) → ChatOllama`	Client Ollama avec `num_ctx` explicite.
`build_chain` `(model, temperature, seed)`	Chaîne LCEL `prompt | llm | StrOutputParser()`, mise en cache par configuration (la V1 la reconstruisait à chaque requête).
`retrieve` `(question, k)`	Appelle `search`.

Fonction	Rôle
`generate` `(question, hits, model, temperature, seed) → str`	Génère une réponse à partir de passages donnés (utilisé par l'évaluation et le DPO).
`below_threshold` `(hits, threshold) → bool`	Vrai si le seuil est actif et que le meilleur score est inférieur.
`answer_question` `(question, k, model, temperature) → dict`	Point d'entrée : recherche, refus par seuil ou génération. Renvoie `answer`, `refused`, `refused_by_threshold`, `retrieved_sources`, `cited_sources`, `contexts`, `hits`, `model`.

La distinction entre `retrieved_sources` (documents dont des passages ont été fournis) et `cited_sources` (documents réellement cités dans la réponse, et présents dans le contexte) corrige un défaut de la V1, qui présentait comme « sources » tous les documents récupérés, cités ou non.

8.3 Les vérifications déterministes

`check_answer(answer, contexts, retrieved_sources)` renvoie un objet `AnswerCheck` :

Champ	Ce qui est vérifié
`refusal`, `clean_refusal`	La réponse est un refus (expressions reconnues par regex : « je ne trouve pas cette information », « le contexte ne mentionne pas »…) ; refus « net » s'il fait au plus 40 mots, c'est-à-dire sans réponse inventée à la suite.
`cited`, `cited_not_retrieved`	Documents cités (regex sur les noms `dbt_NN_…` / `airflow_NN_…`, extension normalisée) ; ceux qui ne faisaient pas partie du contexte.
`quotes`, `unsupported_quotes`	Passages entre guillemets (« », “ ”, " ") d'au moins 4 mots ; ceux qui ne figurent pas mot pour mot dans le contexte, après normalisation (casse, apostrophes, espaces, marqueurs Markdown).
`code_spans`, `unsupported_code`	Identifiants entre accents graves ; ceux absents du contexte (tolère `xcom_push()` pour `xcom_push` et `@task` pour `task`).
propriétés	`has_citation`, `citations_valid`, `quotes_faithful`, `code_support` (part des identifiants présents, 1 s'il n'y en a pas).

Fonctions auxiliaires : `normalize`, `_strip_markdown`, `is_refusal`, `is_clean_refusal`, `cited_documents`, `extract_quotes`, `extract_code_spans`, `_in_contexts`, `_code_in_contexts`. La constante `REFUSAL_MESSAGE` est partagée avec le prompt.

> **POURQUOI DES RÈGLES PLUTÔT QU'UN JUGE**
>
> Ces contrôles sont exacts, reproductibles au bit près, instantanés et gratuits. Ils ne disent pas si la réponse est *bonne*, mais ils détectent de façon fiable les violations les plus graves pour un assistant documentaire : inventer une citation, citer une source qu'on n'a pas lue, inventer un nom de paramètre, refuser à tort. Ils servent deux fois : comme métriques d'évaluation et comme contraintes de la récompense DPO.

CE QU'ON AURAIT PU FAIRE

- **Sorties structurées** : demander une réponse JSON (`{"answer": …, "citations": [{"doc": …, "quote": …}]}`), imposée par le mode `format` d'Ollama. Plus facile à vérifier, mais les petits modèles perdent en qualité de rédaction sous contrainte de format.
- **Citations par identifiant de passage** ([1], [2]) plutôt que par nom de document : plus précis (on sait quel passage justifie quoi).
- **Vérification a posteriori et correction** : supprimer automatiquement les citations non vérifiées avant de renvoyer la réponse. Un garde-fou simple à ajouter dans l'API, qui masque le problème sans le corriger dans le modèle.
- **Few-shot** : ajouter deux exemples de réponses modèles dans le prompt. Souvent efficace avec les petits modèles, au prix de tokens de contexte.

9 API

`api/main.py` (107 lignes) expose le système en HTTP avec FastAPI.

9.1 Points d'entrée

Route	Comportement
`GET /health`	`{"status": "ok" | "degraded", "index_ready": bool, "model": …}`. Utilisée par le healthcheck Docker et par la CD.
`POST /query`	Entrée `{"question": str, "top_k": 1–20}` (Pydantic : question non vide). Sortie : `question`, `answer`, `refused`, `cited_sources`, `retrieved_sources`, `model`, `latency_seconds`.

Erreurs : 422 si l'entrée est invalide (question vide, `top_k` hors de 1–20) ; 400 si la question ne contient que des espaces ; 503 si l'index n'est pas prêt ou si Ollama est injoignable ; 500 pour une erreur de génération. La fonction `lifespan` s'exécute au démarrage : si `chunks.json` existe et que l'index est absent ou obsolète, elle réindexe. Un conteneur neuf avec un volume vide devient ainsi opérationnel sans intervention.

```
curl -X POST http://127.0.0.1:8000/query -H "Content-Type: application/json" \
  -d '{"question": "Comment pousser une valeur dans un XCom ?"}'
```

CE QU'ON AURAIT PU FAIRE

- **Streaming** (Server-Sent Events) : afficher la réponse au fil de la génération ; indispensable pour une interface de chat, inutile pour une API de test.
- **Endpoints asynchrones** avec `ainvoke` : meilleure tenue en charge ; ici, le goulot est le LLM local, qui sert les requêtes en série.
- **Sécurité** : CORS restreint (actuellement `*`), clé d'API, limitation de débit. Nécessaire avant toute exposition publique.
- **Observabilité** : journaliser chaque question, les passages, la réponse et la latence (Langfuse, OpenTelemetry). Ces journaux seraient la meilleure source de questions réelles pour l'évaluation et le DPO.
- **Interface** : un front Streamlit ou Gradio de quelques dizaines de lignes pour une démonstration.

10 Évaluation

Le dossier `evaluation/` mesure la qualité du système et, surtout, dit si une différence entre deux versions est réelle. C'est la partie du projet qui a le plus changé entre la V1 et la V2.

10.1 Le jeu d'évaluation

`eval_dataset.json` contient **85 questions** : 73 dans le périmètre, réparties sur les 10 documents (entre 6 et 10 par document), et 12 hors périmètre. Chaque question du périmètre a une réponse de référence en français, ses documents attendus et des **preuves** : des extraits recopiés mot pour mot du corpus (98 au total).

```
{
  "id": "af_xcom_02",
  "type": "in_scope",
  "question": "Comment pousse-t-on une valeur dans un XCom depuis une tâche Airflow ?",
  "ground_truth": "Avec la méthode xcom_push de l'instance de tâche, par exemple
                   task_instance.xcom_push(key=..., value=...). …",
  "expected_sources": ["airflow_04_xcoms.md"],
  "evidence": ["task_instance.xcom_push(key=",
               "auto-push their results into an XCom key called return_value"]
}
```

Les questions hors périmètre mêlent des sujets éloignés (« Quelle est la capitale de l'Australie ? ») et des **pièges proches du domaine** : Prefect, Dagster, Kafka Connect, l'auto-suspend de Snowflake, le prix de dbt Cloud, NiFi. Ces derniers sont les plus utiles : la recherche ramène des passages qui « ressemblent », et le modèle est tenté de répondre.

`dataset.py` valide le jeu (`validate_dataset`) : identifiants et questions uniques, types connus, documents attendus présents dans le corpus, et **chaque preuve retrouvée dans un des documents attendus** après normalisation. Le test `test_eval_dataset_evidence_is_in_corpus` et un job CI échouent si une preuve disparaît du corpus.

> **POURQUOI DES PREUVES**
>
> En V1, une réponse de référence avait été écrite de mémoire, sur un sujet que le corpus ne couvrait pas ; le système avait correctement refusé, et cette bonne réponse avait été comptée comme une erreur. Les preuves rendent impossible ce type d'erreur, et permettent de mesurer le retrieval sans aucun LLM.

10.2 Les métriques

Retrieval (questions du périmètre, sans LLM)

Métrique	Définition
`evidence_recall`	Part des preuves présentes dans au moins un des 4 chunks récupérés.
`evidence_hit`	1 si au moins une preuve est présente.
`source_hit`	1 si au moins un document attendu figure parmi les documents récupérés.

Métrique	Définition
`mrr`	1 / rang du premier chunk contenant une preuve, 0 si aucun.

Génération, par règles

Métrique	Définition
`answered`	Question du périmètre à laquelle le système a répondu (pas de refus).
`false_refusal`	Refus *alors que la preuve était dans le contexte* : le modèle avait de quoi répondre. Plus bas = mieux.
`has_citation`, `citations_valid`, `cites_expected_source`	Au moins une source citée ; aucune source citée hors contexte ; le document attendu est cité. Calculées sur les réponses non refusées.
`quotes_faithful`	Toutes les citations entre guillemets sont verbatim dans le contexte.
`code_support`	Part des identifiants de code présents dans le contexte.
`correct_refusal`, `clean_refusal`	Questions hors périmètre : le système refuse ; et refuse sans dérouler une réponse inventée.

Génération, par Ragas (questions du périmètre uniquement)

Métrique	Définition
`faithfulness`	Le juge décompose la réponse en affirmations et vérifie chacune contre le contexte : part des affirmations étayées.
`answer_relevancy`	Le juge génère des questions à partir de la réponse ; similarité cosinus moyenne entre ces questions et la question posée.
`context_precision`	Le juge évalue si chaque passage récupéré est utile à la réponse de référence ; précision pondérée par le rang.

Le juge est `qwen2.5:7b`, distinct du générateur. Les embeddings de Ragas sont ceux du projet, avec le préfixe `query:` des deux côtés : answer_relevancy compare deux questions entre elles. La concurrence de Ragas est limitée (`max_workers=2`, `timeout=300` s) parce qu'un Ollama local sert les requêtes une par une ; avec les 16 workers par défaut, une partie des requêtes expirait en V1. Les questions hors périmètre sont **exclues** de Ragas : un refus correct y valait 0 par construction et faisait baisser les moyennes.

10.3 Le déroulement d'un run

`run_eval.py` enchaîne :

1. chargement et validation du jeu (un jeu invalide arrête tout) ;
2. pour chaque question : recherche (mise en cache entre répétitions, car déterministe), métriques de retrieval, puis génération (ou refus par seuil) et métriques par règles ;
3. Ragas, une fois par répétition ;
4. agrégation : moyenne par question sur les répétitions, puis moyenne et intervalle de confiance bootstrap sur les questions ; écart-type entre répétitions si `--repeats` > 1 ;
5. rapport sur les scores du meilleur chunk dans et hors périmètre, avec un seuil de refus suggéré (exactitude équilibrée maximale) ;

6. écriture de `results/<run>/per_question.csv` et `summary.json`, puis journalisation MLflow.

```
python evaluation/run_eval.py --retrieval-only              # quelques secondes, sans LLM
python evaluation/run_eval.py --run-name baseline_v2 --repeats 2
python evaluation/run_eval.py --run-name dpo_v1 --model rag-assistant-dpo
python evaluation/compare_runs.py baseline_v2 dpo_v1
```

Options : `--model` (générateur à évaluer), `--repeats`, `--retrieval-only`, `--skip-ragas`, `--limit` (N premières questions), `--no-mlflow`.

10.4 Comparer deux runs

`compare_runs.py` charge les résultats par question des deux runs, ne garde que les questions communes et applique un **bootstrap apparié** (5 000 tirages) sur les différences B − A. Pour chaque métrique, il affiche les deux moyennes, la différence, son intervalle de confiance, une p-valeur approchée et un verdict : « B meilleur », « B moins bon » ou « non significatif ». Pour `false_refusal`, une baisse est une amélioration.

```
metric             A      B   delta  ci_low  ci_high  p_value  n  verdict
answered       1.000  0.000  -1.000  -1.000   -1.000    0.000 73  B moins bon
correct_refusal 1.000 1.000   0.000   0.000    0.000    …     12  non significatif
```

Extrait simplifié d'une sortie produite par les tests, avec un générateur simulé qui refuse tout : ce ne sont pas des résultats du système.

10.5 Suivi MLflow

`mlflow_tracking.py` enregistre pour chaque run : la configuration **lue dans l'index** (préfixe `index_` : modèle d'embeddings, empreinte du corpus, paramètres de chunking), le générateur, le juge, top-k, candidats, reranker, seuil, `num_ctx`, nombre de répétitions, empreinte du jeu d'évaluation, mode ; les métriques avec leurs bornes d'intervalle ; les tags `git_commit` et `git_dirty` ; et en artefacts le CSV, le résumé et le jeu d'évaluation. En V1, la taille de chunk enregistrée était celle du `.env` au moment de l'évaluation, pas forcément celle de l'index réellement interrogé.

`_ragas_compat.py` contourne un import cassé : Ragas 0.4.3 importe un module VertexAI que les versions récentes de `langchain-community` ont supprimé. Le module enregistre un module factice avant l'import de Ragas ; sans effet fonctionnel puisque le projet n'utilise pas VertexAI.

10.6 Fonctions

Fonction	Rôle
`dataset.load_eval_dataset`, **`load_corpus_texts`**	Chargement du jeu et du corpus.
`dataset.validate_dataset` `(items, corpus) → [erreurs]`	Validation complète ; liste vide si tout est correct.
`dataset.norm`, **`evidence_in`**, **`dataset_fingerprint`**	Normalisation partagée avec `checks.py` ; test de présence d'une preuve ; empreinte du fichier.

Fonction	Rôle
metrics.retrieval_metrics `(item, hits) → dict`	Les quatre métriques de retrieval.
metrics.generation_metrics `(item, answer, hits, evidence_hit) → dict`	Métriques par règles (via `check_answer`), selon le type de question.
metrics.bootstrap_ci `(values, n_boot=2000) → (moyenne, bas, haut, n)`	Intervalle percentile, valeurs manquantes ignorées.
metrics.paired_bootstrap `(a, b, n_boot=5000) → dict`	Différence appariée, intervalle, p-valeur.
metrics.suggest_threshold `(in_scores, out_scores) → dict`	Seuil qui maximise l'exactitude équilibrée.
run_eval.run_pipeline, **run_ragas**, **summarize**, **print_summary**, **retrieval_report**, **safe_name**, **main**	Étapes du run décrites ci-dessus.
compare_runs.load_run, **compare**, **main**	Chargement d'un run par nom ou chemin ; tableau comparatif.
mlflow_tracking.git_state, **init_mlflow**, **log_eval_run**	Commit et état du dépôt ; initialisation ; journalisation d'un run.

10.7 Choix et alternatives

CE QU'ON AURAIT PU FAIRE

- **Un juge plus fort** : un modèle de 70B en local (trop gros pour un GPU de 8 Go) ou une API (GPT-4o, Claude) pour Ragas. Scores plus fiables, au prix d'un coût et d'une dépendance externe. C'est l'amélioration la plus rentable si un budget existe.
- **Autres frameworks** : DeepEval, TruLens, ARES, ou les évaluateurs de LangSmith. Mêmes familles de métriques, avec leurs propres biais de juge.
- **Évaluation humaine** : noter 30 réponses à la main (exactitude, complétude) et mesurer l'accord avec le juge. La seule façon de savoir si le juge est fiable sur ce domaine.
- **Exactitude de la réponse contre la référence** (answer correctness de Ragas, ou un score sémantique) : la réponse de référence existe déjà dans le jeu mais n'est utilisée que par context_precision.
- **Jeu plus grand** : 85 questions donnent des intervalles d'environ ±0,05 à ±0,10 sur des métriques binaires. Pour détecter des écarts de 0,03, il en faudrait plusieurs centaines.

11 Alignement par DPO

Le dossier `training/` améliore le générateur par apprentissage de préférences. Le pipeline compte quatre étapes, dont trois tournent en local avec Ollama et une sur GPU (Colab ou Kaggle).

11.1 Objectif et principe

Le DPO vise les défauts du générateur que l'on sait mesurer : citations entre guillemets qui ne sont pas verbatim, documents cités hors contexte, identifiants de code inventés, refus à tort, réponse au lieu de refus hors périmètre, réponses peu ancrées. Le principe : faire produire au modèle actuel plusieurs réponses à une même question, les noter avec une **récompense vérifiable**, et lui apprendre à préférer la meilleure.

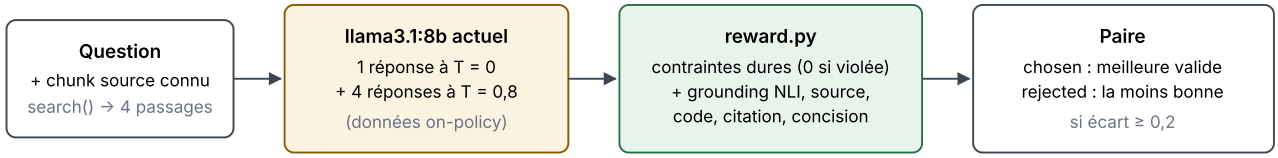


Figure 4. Construction d'une paire de préférences (`build_preferences.py`).

11.2 Étape 1 : les questions d'entraînement

`generate_questions.py` demande au modèle juge (`qwen2.5:7b`, température 0,7, sortie JSON) de rédiger 2 questions en français par chunk d'au moins 60 tokens, en gardant l'identifiant du chunk source. S'y ajoutent les 60 questions hors périmètre de `out_of_scope_train.json`, différentes de celles de l'évaluation. Puis :

- **déduplication** (`dedupe`) : questions trop proches entre elles (Jaccard sur les mots porteurs de sens ≥ 0,8) ;
- **filtre anti-fuite** (`LeakageFilter`) : toute question trop proche d'une question d'évaluation est écartée, par recouvrement lexical (Jaccard ≥ 0,5) ou par similarité d'embeddings au-delà d'un seuil calibré sur le jeu d'évaluation (0,949 ici ; un seuil fixe de 0,88 écartait presque tout, section 19.5).

Fonctions : `content_tokens`, `jaccard`, `product_of`, `generate_for_chunk`, `LeakageFilter.keep`, `dedupe`, `main`. Sortie : `training/data/questions.jsonl`.

11.3 Étape 2 : la récompense et les paires

`reward.py` définit `compute_reward(answer, question_type, hits, source_document, nli)` :

Cas	Récompense
Hors périmètre	0 si le modèle répond (`answered_out_of_scope`) ; 1 pour un refus net ; 0,5 pour un refus suivi d'un développement.
Périmètre, contraintes dures	0 en cas de refus (`false_refusal`, le chunk source était dans le contexte), réponse de moins de 12 mots (`too_short`), document cité hors contexte, citation inventée.

Cas	Récompense
Périmètre, composantes	Moyenne pondérée : grounding NLI 0,40 ; document source cité 0,20 ; identifiants de code présents 0,15 ; au moins une citation 0,10 ; concision 0,15 (1 jusqu'à 180 mots, décroît jusqu'à 0 à 400). Les poids sont renormalisés si le NLI est désactivé.

Le **grounding** (`grounding_score`) découpe la réponse en phrases (`answer_sentences`), calcule pour chacune la probabilité d'implication par le meilleur passage du contexte (`NliScorer.entailment`), et fait la moyenne. Le modèle NLI est chargé une fois (`get_nli_scorer`).

`build_preferences.py` applique la récompense : pour chaque question, il recherche les passages ; si c'est une question du périmètre dont le chunk source n'a *pas* été récupéré, il l'écarte (on ne saurait pas si un refus serait correct) ; il génère une réponse greedy et 4 échantillonnées (`sample_answers`, doublons exacts retirés), les note, et forme une paire si la meilleure réponse valide dépasse la moins bonne d'au moins 0,2 (`select_pair`). Sortie : `preferences.jsonl` (prompt complet, chosen, rejected, détails des récompenses) et `preferences_stats.json` (questions écartées, motifs de rejet, questions sans paire).

> **POURQUOI UNE RÉCOMPENSE VÉRIFIABLE**
> Entraîner contre un juge LLM de 7-8B, c'est optimiser un signal bruité et biaisé : le modèle apprend à plaire au juge plutôt qu'à mieux répondre (*reward hacking*). Les contraintes dures ne peuvent pas être trompées : une citation est dans le contexte ou ne l'est pas. Le grounding utilise un modèle NLI dédié, distinct du générateur et du juge d'évaluation. Et l'évaluation finale emploie Ragas, que l'entraînement n'a jamais vu : une amélioration qui n'apparaîtrait que sur les métriques de la récompense serait suspecte.

> **POURQUOI DES DONNÉES ON-POLICY**
> Les réponses viennent du modèle qu'on veut améliorer, dans les conditions réelles (mêmes passages, même prompt). Le DPO corrige alors ses erreurs effectives, pas celles d'un autre modèle. Des paires construites avec un modèle plus fort comme « chosen » apprendraient un style que le modèle 8B ne sait pas reproduire.

11.4 Étape 3 : l'entraînement

`train_dpo.py`, lancé depuis `dpo_colab.ipynb` sur un GPU de 16 Go :

- **modèle de base** : `meta-llama/Llama-3.1-8B-Instruct`, le même que `llama3.1:8b` dans Ollama, pour que l'adaptateur s'applique au modèle déployé (accès Hugging Face à demander) ;
- **QLoRA** : chargement en 4 bits NF4 avec double quantification ; LoRA r = 16, α = 32, dropout 0,05, sur les 7 projections (`q, k, v, o, gate, up, down`) ; calcul en bf16 si le GPU le supporte nativement (Ampere et suivants), sinon fp16 (T4) ;
- **DPO** (TRL) : β = 0,1, taux d'apprentissage $5{\cdot}10^{-6}$, planification cosinus avec 10 % de warm-up, 1 époque, batch effectif 8 (1 × 8 d'accumulation), gradient checkpointing, 10 % des paires en validation ;
- **gabarit de chat** : celui qu'Ollama applique à Llama 3.1 (`OLLAMA_LLAMA3_CHAT_TEMPLATE`), et non celui de Hugging Face, qui ajoute des lignes de date au message système : on entraîne sur le format exact de

l'inférence ;

- **longueur** : les paires de plus de 3 072 tokens sont écartées, pas tronquées (une troncature couperait la fin de la réponse, c'est-à-dire ce qu'on optimise).

Fonctions : `load_pairs`, `sequence_length`, `split_pairs` (découpage reproductible), `prepare_tokenizer`, `build_trainer` (assemble `DPOConfig` et `DPOTrainer`, utilisé aussi par le test), `summarize_logs`, `parse_args`, `main`. Sortie : `training/outputs/dpo-lora/` (adaptateur, tokenizer, `training_summary.json` avec l'exactitude et la marge des récompenses implicites sur la validation).

11.5 Étape 4 : retour dans Ollama et décision

`export_ollama.py` propose deux voies :

- **adaptateur** (par défaut) : copie de l'adaptateur et Modelfile `FROM llama3.1:8b` + `ADAPTER ./adapter`, avec `num_ctx` et les jetons d'arrêt de Llama 3.1 ; puis `ollama create rag-assistant-dpo`. Rapide, mais l'adaptateur entraîné sur une base NF4 est appliqué à une base Q4_K_M ; la documentation d'Ollama prévient que le résultat peut s'écarter ;
- **fusion** (`--merge`) : fusion de l'adaptateur dans les poids fp16 (environ 16 Go, 32 Go de RAM conseillés : Kaggle oui, Colab gratuit non), puis `ollama create --quantize q4_K_M`. À utiliser si la première voie dégrade les résultats.

Fonctions : `adapter_modelfile`, `merged_modelfile`, `export_adapter`, `export_merged`, `main`.

```
FROM llama3.1:8b
ADAPTER ./adapter
PARAMETER num_ctx 8192
PARAMETER stop "<|eot_id|>"
PARAMETER stop "<|start_header_id|>"
```

> **RÈGLE DE DÉCISION**
>
> Le modèle DPO n'est adopté que si `compare_runs.py` montre une amélioration significative d'au moins une métrique visée (`quotes_faithful`, `faithfulness`, `false_refusal`, `citations_valid`) **et** aucune dégradation significative de `answer_relevancy` ni de `correct_refusal`. Avant tout entraînement, si la baseline montre ces métriques déjà au plafond, le DPO n'a rien à corriger : on s'arrête.

ATTENTION

- La récompense ne mesure pas si la réponse *répond* à la question. Risque : un modèle qui apprend des réponses fidèles mais évasives. Garde-fous : longueur minimale, answer_relevancy dans la règle de décision, lecture de 30 paires avant d'entraîner.
- **Seule l'étape 1 a été menée à terme ; l'étape 2 a été interrompue** (chapitre 19). En CI, l'entraînement ne peut pas tourner : depuis la version 1.15, TRL calcule les log-probabilités avec un noyau Triton qui exige un GPU CUDA. Le test CPU vérifie ce qui précède (données, gabarit, tokenisation, LoRA). Le premier run Colab, avec `--limit 20`, est le vrai test.
- Sur T4 (fp16), TRL convertit les adaptateurs en bf16 pour les modèles quantifiés : c'est le point le plus susceptible de poser problème.

11.6 Choix et alternatives

CE QU'ON AURAIT PU FAIRE

- **Ne pas entraîner du tout** : améliorer le prompt (few-shot), ajouter un post-traitement qui supprime les citations non vérifiées, changer de générateur (Qwen 2.5 7B, Mistral). Moins coûteux, à essayer d'abord si la baseline révèle peu de violations.
- **SFT** (fine-tuning supervisé) sur les meilleures réponses seulement (rejection sampling / best-of-n) : plus simple que DPO, mais n'apprend pas explicitement ce qu'il ne faut pas faire.
- **Variantes de DPO** : IPO (plus robuste au surapprentissage sur des paires faciles), KTO (accepte des réponses notées bonnes/mauvaises sans paires), ORPO (combine SFT et préférences sans modèle de référence), SimPO (sans référence, normalisé par la longueur). Toutes disponibles dans TRL.
- **RL en ligne** : PPO ou GRPO avec la même récompense vérifiable, appelée pendant l'entraînement. C'est l'approche des modèles de raisonnement récents ; plus puissante (le modèle explore), mais bien plus coûteuse en calcul et délicate à stabiliser.
- **Bandit contextuel sur le retrieval** : apprendre quand augmenter top-k, activer le reranker ou réécrire la requête. Léger, mais gain faible sur un corpus de 10 documents.
- **Unsloth** pour l'entraînement : plus rapide et plus économe en mémoire sur Colab, avec export GGUF intégré vers Ollama.

12 Tests

52 tests répartis en 9 fichiers, conçus pour tourner en quelques secondes sans modèle, sans base vectorielle réelle ni GPU, sauf deux d'entre eux qui demandent ChromaDB ou PyTorch.

Fichier	Tests	Ce qui est vérifié
`test_build_corpus.py`	5	Rôles RST, titres, directives, commentaires, inclusions d'exemples ; composants MDX (constantes, termes, fichiers, FAQ, fragments avec props) ; suppression des sections de navigation. Sources simulées.
`test_chunking.py`	4	Sections et `#` dans le code ; regroupement, recoupage et fil d'Ariane ; tous les chunks du vrai corpus sous la taille maximale ; empreinte du corpus.
`test_checks.py`	7	Détection des refus, normalisation des citations de documents, citations verbatim et inventées, identifiants de code.
`test_dataset.py`	4	Toutes les preuves sont dans le corpus ; composition du jeu ; rejet d'une preuve inventée et des doublons.
`test_metrics.py`	7	Métriques de retrieval (rang, rappel), faux refus, bootstrap, test apparié (effet détecté, absence d'effet), seuil.
`test_pipeline.py`	4	Run d'évaluation complet avec recherche et LLM simulés ; mode retrieval seul ; détection d'une régression par `compare_runs` ; API (`/health`, `/query`, validation).
`test_vector_store.py`	3	Indexation ChromaDB réelle avec embeddings factices : métadonnées, recherche, détection d'un index obsolète ou d'un autre modèle, reranking.
`test_training.py`	16	Récompense (cas valides, 4 contraintes dures, hors périmètre, composantes, NLI simulé), sélection des paires, disjonction et filtre anti-fuite, déduplication, découpage, export Ollama, gabarit de chat identique à celui d'Ollama.
`test_train_dpo_smoke.py`	2	Avec un Llama minuscule : tokenisation par TRL (BOS en tête, `<|eot_id|>` en fin de réponse), seuls les adaptateurs LoRA entraînables ; deux pas d'entraînement et sauvegarde de l'adaptateur, uniquement si un GPU est présent.

POURQUOI DES FONCTIONS INJECTABLES

Les fonctions qui dépendent d'un service externe prennent la dépendance en paramètre (`fetch` pour le téléchargement, `count_tokens` pour le tokenizer, `nli` pour le modèle NLI) ou sont remplacées par `monkeypatch` dans les tests (`search`, `generate`). C'est ce qui permet de tester la logique en isolation, rapidement, en CI.

CE QU'ON AURAIT PU FAIRE

- **Tests d'intégration avec de vrais modèles** (embeddings et reranker sur CPU, un petit LLM via Ollama dans la CI) : plus lents, ils vérifieraient les tailles réelles en tokens et la qualité minimale du retrieval.
- **Seuil de non-régression du retrieval en CI** : échouer si `evidence_hit` passe sous une valeur de référence. Nécessite les modèles d'embeddings en CI.
- **Couverture de code** (pytest-cov) affichée en CI.

13 Conteneurisation et CI/CD

L'API est livrée sous forme d'image Docker ; GitHub Actions vérifie chaque modification et publie l'image.

13.1 Docker

Le `Dockerfile` part de `python:3.12-slim`, installe PyTorch en version CPU (les embeddings et le reranker tournent sur CPU dans le conteneur ; le LLM tourne dans Ollama sur l'hôte, avec le GPU), puis les dépendances, copie le code et **construit les chunks au build**. Au démarrage, l'API indexe si nécessaire. Un `HEALTHCHECK` interroge `/health`.

`docker-compose.yml` publie l'API sur le port 8600 (le 8000 peut être réservé par Windows avec Hyper-V), monte `./chroma_db` en volume, met en cache les modèles Hugging Face dans un volume nommé, et contacte Ollama sur l'hôte via `host.docker.internal`. Le `.dockerignore` exclut les tests, l'entraînement, les résultats et les secrets.

13.2 CI/CD

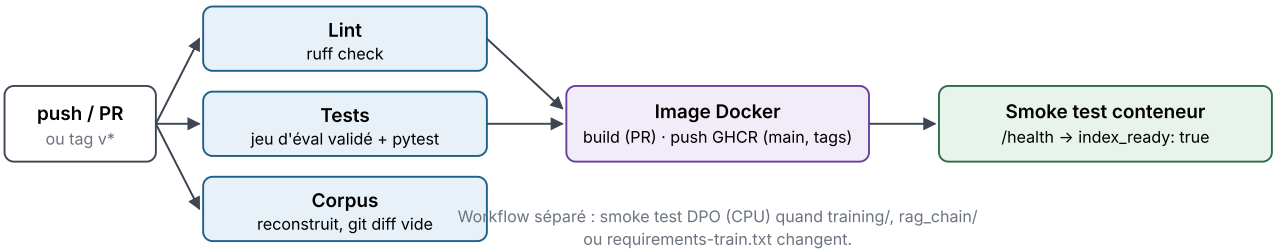


Figure 5. Workflow `.github/workflows/ci-cd.yml`.

Le job « Corpus » mérite une explication : il relance `build_corpus.py` sur le runner et exige que `corpus/` soit identique au contenu versionné. Il garantit que le corpus n'a pas été modifié à la main et qu'il est reproductible depuis les sources figées. Le smoke test du conteneur garantit que l'image publiée démarre, télécharge ses modèles, construit son index et répond.

Tags d'image produits : SHA court, nom de branche, version sémantique pour les tags `v*`, et `latest` sur `main`. Le paquet GHCR est créé en privé ; on peut le rendre public dans les réglages du paquet.

CE QU'ON AURAIT PU FAIRE

- **Évaluation en CI** : un run `--retrieval-only` sur chaque PR, avec commentaire automatique des métriques et comparaison avec `main`. Demande de télécharger les modèles d'embeddings (cache possible).
- **Déploiement continu** vers un hébergeur (Fly.io, Render, Cloud Run). Bloqué ici par le LLM : il faudrait un GPU ou une API.
- **pre-commit** en local (ruff, fin de ligne) pour attraper les erreurs avant le push.
- **Dependabot** pour les mises à jour des actions et des dépendances (et la fin du support de Node 20 dans les actions `checkout` / `setup-python` v4/v5).

PARTIE IV

Prendre du recul

D'où vient la V2, quelles décisions la structurent, ce qui reste fragile ou non mesuré, comment la faire évoluer, et comment l'utiliser pas à pas.

14 Historique : de la V1 à la V2

La V1 fonctionnait et était bien documentée. Sa relecture critique a montré que plusieurs de ses conclusions n'étaient pas étayées par ses propres mesures. Cette relecture est la meilleure leçon du projet.

14.1 Ce que mesurait la V1

Configuration : corpus PDF généré depuis le HTML des sites, `all-MiniLM-L6-v2`, chunks fixes de 500 tokens (tiktoken) avec 50 de chevauchement, top-4, `llama3.1:8b` comme générateur *et* comme juge, 15 questions dont une hors périmètre.

Configuration	Chunks	Faithfulness	Answer relevancy	Context precision
500 / 50 (retenue)	126	0,818	0,581	0,987
300 / 30	221	0,748	0,629	0,963

14.2 Ce que la relecture a montré

Constat	Explication	Réponse en V2
Modèle d'embeddings non multilingue	Questions en français, corpus en anglais, modèle entraîné sur de l'anglais. L'answer relevancy basse avait été attribuée au seul juge ; le modèle d'embeddings, utilisé aussi par Ragas pour cette métrique, était un suspect au moins aussi sérieux, jamais testé.	`multilingual-e5-small` avec préfixes.
« Retrieval excellent » sur-affirmé	0,987 de context precision sur 10 documents, avec des questions qui nomment l'outil, ne prouve presque rien. La question hors périmètre obtenait même 1,0, signe que la métrique jugée par LLM n'était pas fiable ici.	Métriques de retrieval contre des preuves, sans LLM.
Comparaison 500 vs 300 non concluante	15 questions, une exécution par configuration ; des questions passaient de 1,0 à 0,0 d'un run à l'autre. Un écart de −0,07 est dans le bruit (intervalle d'environ ±0,2).	85 questions, intervalles de confiance, bootstrap apparié, répétitions.
Refus correct compté comme échec	La question hors périmètre valait 0 par construction et faisait passer la faithfulness moyenne de 0,877 à 0,818.	Refus mesuré à part ; Ragas sur le périmètre seulement.
Corpus bruité	Artefacts de navigation dans le texte, phrases coupées par l'extraction HTML → PDF → texte.	Corpus reconstruit depuis les sources.
Étiquetage MLflow erroné possible	Paramètres lus dans le `.env` au moment de l'évaluation, pas dans l'index réellement interrogé.	Configuration stockée dans l'index et relue.
Juge = générateur	Biais d'auto-évaluation reconnu dans le README de la V1.	Juge `qwen2.5:7b`.
Troncature silencieuse possible	Fenêtre de contexte Ollama par défaut non fixée.	`num_ctx = 8192`.
Sources affichées = sources récupérées	L'API présentait tous les documents récupérés comme sources.	Sources citées et sources récupérées distinguées.

LA LEÇON

Une métrique n'est utile que si l'on sait ce qu'elle mesure, avec quelle précision, et ce qu'elle ne mesure pas. Avant d'interpréter un écart, il faut se demander s'il dépasse le bruit ; avant d'attribuer une cause, il faut avoir éliminé les autres suspects. C'est aussi la condition pour que le DPO ait un sens : on ne peut pas optimiser un signal qu'on ne sait pas mesurer.

15 Récapitulatif des décisions

Toutes les décisions structurantes, avec l'alternative principale écartée et la raison.

Sujet	Choix	Alternative principale	Raison du choix
Source du corpus	Fichiers sources MDX / RST à versions figées	HTML des sites (V1), build Sphinx/Docusaurus	Texte propre, structure conservée, reproductible au caractère près.
Format du corpus	Markdown	PDF (V1)	Le PDF n'apportait rien et dégradait le texte.
Chunking	Par sections + fil d'Ariane, recoupage au-delà de 400 tokens	Taille fixe (V1), sémantique, parent-child	La structure existe déjà ; gratuit, déterministe.
Unité de taille	Tokens du modèle d'embeddings	tiktoken cl100k (V1)	C'est la limite réelle d'encodage (512).
Embeddings	`multilingual-e5-small`	bge-m3, e5-large, API	Multilingue, CPU, gratuit ; à comparer via `--retrieval-only`.
Recherche	20 candidats cosinus → cross-encodeur → 4	Hybride BM25, HyDE	Gain de précision standard, coût faible.
Base vectorielle	ChromaDB persistante	Qdrant, pgvector, FAISS	Sans serveur ; métadonnées de collection pour la traçabilité.
Intégrité de l'index	Configuration stockée et vérifiée	Aucun contrôle (V1)	Empêche les résultats absurdes silencieux.
Générateur	`llama3.1:8b` local	API commerciale, Qwen 2.5	Gratuit, local ; base du DPO.
Prompt	Règles vérifiables, définition unique	Prompt libre, sortie JSON	Rend mesurable le respect des règles et réutilisable pour l'entraînement.
Refus hors périmètre	Par le LLM ; seuil optionnel calibré	Seuil obligatoire	Un faux rejet coûte une réponse ; le seuil n'est activé que s'il est calibré.
Jeu d'évaluation	85 questions avec preuves validées	15 questions écrites de mémoire (V1)	Ancrage vérifié, puissance statistique, retrieval mesurable.
Métriques	Règles + Ragas (juge distinct)	Ragas seul, juge = générateur (V1)	Signal exact là où c'est possible, sémantique ailleurs.
Statistiques	IC bootstrap, test apparié, répétitions	Moyennes brutes (V1)	Distinguer un effet du bruit.
Méthode d'alignement	DPO hors ligne	PPO / GRPO en ligne, SFT	Simple, stable, faisable sur un GPU gratuit.
Récompense	Vérifiable (règles + NLI)	Juge LLM	Résiste au reward hacking.
Données DPO	On-policy, questions synthétiques filtrées	Réponses d'un modèle plus fort	Corrige les erreurs réelles du modèle déployé.
Entraînement	QLoRA, gabarit Ollama, prompt identique	Fine-tuning complet	Tient sur 16 Go ; cohérence entraînement / inférence.

Sujet	Choix	Alternative principale	Raison du choix
Retour en production	Adaptateur dans Ollama, fusion en repli	Servir avec vLLM / transformers	Reste dans la stack existante.
Tests	Sans modèle, dépendances injectées	Tests d'intégration lourds	Rapides, exécutables en CI gratuite.
CI/CD	GitHub Actions + GHCR	GitLab CI, Docker Hub	Intégré au dépôt, gratuit.

16 Limites et points non mesurés

16.1 Ce qui n'a pas encore été mesuré

- **Les résultats de la V2 sont mesurés** (chapitre 19), mais pour une seule configuration : aucune variante (autre modèle d'embeddings, sans reranker, autre taille de chunk) n'a encore été comparée à la baseline.
- **Le pipeline DPO après l'étape 2** : entraînement sur GPU, export vers Ollama, réévaluation et comparaison appariée avec `baseline_v2`.
- **La fiabilité du juge** `qwen2.5:7b` sur ce domaine : il est différent du générateur, pas forcément meilleur. Sur un essai de deux questions relancé à l'identique, la faithfulness est passée de 0,75 à 1,0, et une réponse du juge est restée illisible aux deux répétitions de la baseline.
- **L'accord avec un humain** : aucune réponse n'a été annotée à la main.

16.2 Limites de conception

- **Corpus restreint** : 10 pages, un seul domaine. Les questions transverses (qui demandent plusieurs documents) sont peu représentées.
- **Taille du jeu** : 85 questions donnent des intervalles d'environ ±0,05 à ±0,10 sur les métriques binaires ; les petites améliorations resteront non significatives.
- **Pertinence non mesurée par la récompense DPO** : seulement l'ancrage et le respect des règles.
- **Questions d'entraînement synthétiques** : moins variées que de vraies questions d'utilisateurs.
- **Adaptateur QLoRA appliqué à une base Q4_K_M** : écart possible entre entraînement et inférence.
- **Convertisseurs de corpus spécialisés** : ajouter une page demande de relire la sortie.
- **API non sécurisée** (CORS ouvert, pas d'authentification) : acceptable en local uniquement.
- **Détection des refus par expressions régulières** : un refus formulé autrement ne serait pas reconnu.

17 Pistes d'évolution

Classées par rapport gain / effort. Chacune peut être mesurée avec le protocole existant.

#	Évolution	Gain attendu	Effort
1	Mesurer la baseline V2 (`--repeats 2`)	**Fait** : résultats et lecture au chapitre 19.	Fait
2	Comparer 2-3 modèles d'embeddings et rerankers en `--retrieval-only`	Choix fondé sur des mesures, en quelques minutes.	Faible
3	Recherche hybride BM25 + vecteurs (fusion RRF)	Meilleur rappel sur les identifiants exacts.	Moyen
4	Annoter 30 réponses à la main	Mesurer l'accord du juge avec un humain ; calibrer la confiance dans Ragas.	Moyen
5	Calibrer et activer `RELEVANCE_THRESHOLD`	Calibré (section 19.4) : à ne pas activer en l'état.	Faible
6	Parent-child retrieval	Plus de contexte pour le LLM sans perdre en précision de recherche.	Moyen
7	Pipeline DPO complet (petit run, puis run complet)	Moins de citations inventées et de faux refus, si la baseline en montre.	En cours
8	Journaliser les questions de l'API	**Fait** : traces Langfuse (section 20.1).	Fait
9	Agrandir le corpus (sources, snapshots, Airflow TaskFlow…)	Couverture ; questions transverses.	Moyen
10	Interface Streamlit / Gradio, streaming	**Fait** : interface web de test (section 20.2).	Fait

18 Mode d'emploi

18.1 Installation

```
python -m venv venv
venv\Scripts\activate                # Windows
pip install -r requirements.txt
cp .env.example .env
ollama pull llama3.1:8b
ollama pull qwen2.5:7b
```

Si le port 11434 est réservé par Windows (Hyper-V, WSL2) : `OLLAMA_HOST=127.0.0.1:12500 ollama serve`, et reporter le port dans `OLLAMA_BASE_URL`. Vérifier les plages réservées avec `netsh interface ipv4 show excludedportrange protocol=tcp`.

Pour vérifier le code : `pip install -r requirements-dev.txt`, puis `ruff check .` et `pytest -q`.

18.2 Construire et interroger

```
python ingestion/build_corpus.py      # optionnel : le corpus est versionné
python ingestion/chunking.py
python retrieval/vector_store.py      # indexation + 4 questions témoins
python rag_chain/chain.py             # deux questions de démonstration
uvicorn api.main:app --reload --port 8000   # Swagger : http://127.0.0.1:8000/docs
```

Avec Docker : `docker compose up -d --build`, puis `curl http://127.0.0.1:8600/health`.

18.3 Évaluer

```
python evaluation/dataset.py                       # valider le jeu
python evaluation/run_eval.py --retrieval-only
python evaluation/run_eval.py --run-name baseline_v2 --repeats 2
python evaluation/compare_runs.py baseline_v2 autre_run
mlflow ui --backend-store-uri sqlite:///mlruns.db   # http://127.0.0.1:5000
```

18.4 Entraîner par DPO

```
python training/generate_questions.py --per-chunk 2
python training/build_preferences.py --samples 4 --limit 20   # essai
python training/build_preferences.py --samples 4              # complet
# lire 30 paires de training/data/preferences.jsonl
# Colab / Kaggle : training/dpo_colab.ipynb → dpo-lora.zip
python training/export_ollama.py --adapter training/outputs/dpo-lora
ollama create rag-assistant-dpo -f training/outputs/ollama/Modelfile
python evaluation/run_eval.py --run-name dpo_v1 --model rag-assistant-dpo --repeats 2
python evaluation/compare_runs.py baseline_v2 dpo_v1
```

19 Résultats mesurés et état d'avancement

Ce chapitre rend compte de la première exécution réelle de la V2, les 12 et 13 octobre 2026, sur une machine Windows avec Ollama : la mise en route, les résultats de la baseline, l'analyse des échecs, les défauts que seule l'exécution a révélés, et l'état du pipeline DPO. Tous les chiffres viennent de `evaluation/results/baseline_v2/` (commit `4153245`) et du run MLflow associé.

19.1 Mise en route : une configuration restée en V1

Le code était prêt, la machine ne l'était pas. Le fichier `.env`, non versionné, datait de la V1 et contredisait `.env.example` sur trois lignes :

Variable	Valeur trouvée	Valeur V2	Conséquence observée
`OLLAMA_BASE_URL`	port 12600	port 11434	Connexion refusée (*WinError 10061*) à la première génération : Ollama écoutait sur 11434.
`EMBEDDING_MODEL_NAME`	all-MiniLM-L6-v2	multilingual-e5-small	Index non multilingue ; 110 chunks sur 166 tronqués à l'encodage (limite de 256 tokens).
`CHUNK_SIZE_TOKENS`	500	400	Chunks plus longs que ce que le modèle d'embeddings sait encoder.

S'y ajoutaient deux modèles Ollama absents (`llama3.1:8b` et `qwen2.5:7b`), un index ChromaDB hérité de la V1 (126 chunks, sans métadonnées) et l'absence de `chunks.json`. Deux garde-fous du chapitre 7 ont joué leur rôle : `ensure_index_ready` a refusé l'ancien index au lieu de servir des résultats issus d'un autre corpus, et `count_truncated` a signalé les 110 chunks tronqués. Après correction, le chunking produit **207 chunks** (médiane 311 tokens, maximum 400), dont aucun n'est tronqué.

> CE QU'ON AURAIT PU FAIRE
>
> **Valider la configuration au démarrage** : une classe `pydantic-settings` et un contrôle explicite (Ollama joignable, modèles présents, modèle d'embeddings cohérent avec l'index) auraient remplacé une trace d'erreur de 80 lignes par un message d'une ligne. Le fichier `.env` n'étant pas versionné, rien ne le fait évoluer avec le code : c'est le point faible de toute configuration par variables d'environnement.

19.2 La baseline V2

Commande : `python evaluation/run_eval.py --run-name baseline_v2 --repeats 2`. Conditions : 85 questions (73 dans le périmètre, 12 hors périmètre), générateur `llama3.1:8b`, juge `qwen2.5:7b`, index `multilingual-e5-small` de 207 chunks, 20 candidats puis reranking, 4 passages transmis, seuil de refus désactivé. Chaque répétition demande 85 générations (latence médiane 2,38 s) puis environ 26 minutes de jugement Ragas.

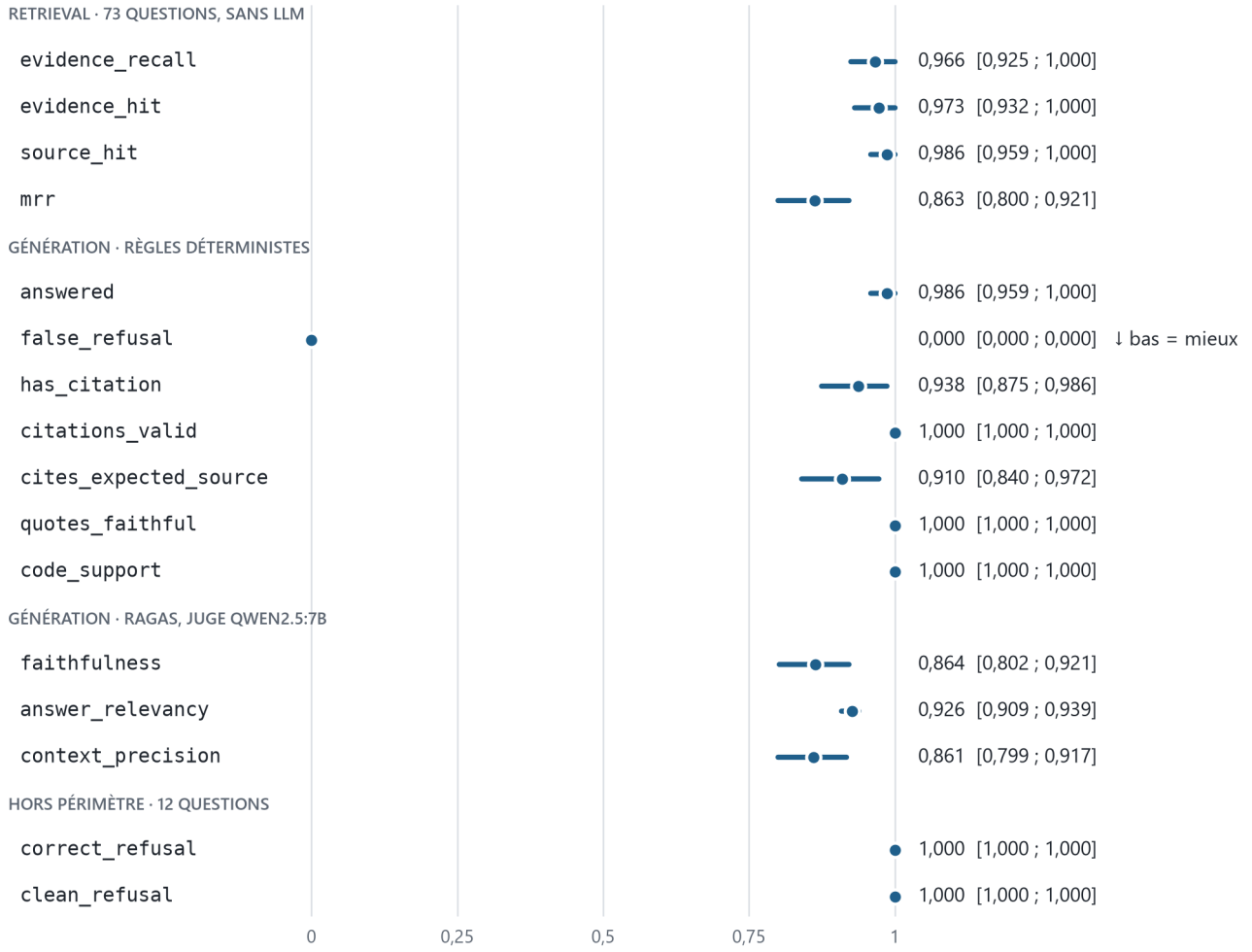


Figure 6. Baseline V2 : moyenne (point) et intervalle de confiance bootstrap à 95 % (trait) de chaque métrique, sur les questions. Moyenne de 2 répétitions par question.

Métrique	Ce qu'elle mesure	Moyenne	IC à 95 %	n	σ répét.
`evidence_recall`	Part des preuves présentes dans les 4 chunks	**0,966**	[0,925 ; 1,000]	73	0,000
`evidence_hit`	Au moins une preuve dans les 4 chunks	**0,973**	[0,932 ; 1,000]	73	0,000
`source_hit`	Document attendu parmi les récupérés	**0,986**	[0,959 ; 1,000]	73	0,000
`mrr`	1 / rang du premier chunk contenant une preuve	**0,863**	[0,800 ; 0,921]	73	0,000
`answered`	Question du périmètre avec réponse	**0,986**	[0,959 ; 1,000]	73	0,000
`false_refusal`	Refus alors que la preuve était là (plus bas = mieux)	**0,000**	[0,000 ; 0,000]	73	0,000
`has_citation`	Au moins une source citée	**0,938**	[0,875 ; 0,986]	72*	0,010
`citations_valid`	Aucune source citée hors contexte	**1,000**	[1,000 ; 1,000]	72*	0,000

Métrique	Ce qu'elle mesure	Moyenne	IC à 95 %	n	σ répét.
`cites_expected_source`	Le document attendu est cité	**0,910**	[0,840 ; 0,972]	72*	0,010
`quotes_faithful`	Citations entre guillemets verbatim	**1,000**	[1,000 ; 1,000]	72*	0,000
`code_support`	Identifiants de code présents dans le contexte	**1,000**	[1,000 ; 1,000]	72*	0,000
`faithfulness`	Affirmations étayées par le contexte (juge)	**0,864**	[0,802 ; 0,921]	72*	0,005
`answer_relevancy`	La réponse répond à la question (juge)	**0,926**	[0,909 ; 0,939]	73	0,008
`context_precision`	Passages utiles bien classés (juge)	**0,861**	[0,799 ; 0,917]	73	0,002
`correct_refusal`	Hors périmètre : le système refuse	**1,000**	[1,000 ; 1,000]	12	0,000
`clean_refusal`	Hors périmètre : refus sans développement	**1,000**	[1,000 ; 1,000]	12	0,000

* Une question sans valeur : celle restée sans réponse pour les métriques de citation, celle dont le jugement est illisible pour la faithfulness.

La colonne *σ répét.* est l'écart-type entre les deux répétitions : nul pour le retrieval, qui est déterministe, et inférieur à 0,01 partout ailleurs. L'incertitude vient donc presque entièrement de l'échantillon de questions, pas du hasard d'une exécution.

19.3 Lecture des résultats

- **Le retrieval n'est pas le facteur limitant.** Une preuve figure dans les 4 passages pour 97 % des questions, et le bon passage arrive le plus souvent en tête (MRR 0,863).
- **Le générateur respecte les règles vérifiables.** Aucun faux refus, aucune source citée hors contexte, aucune citation entre guillemets inventée, aucun identifiant de code absent du contexte.
- **La marge se situe dans l'ancrage et les citations.** La faithfulness vaut 0,864, 6 % des réponses ne citent aucune source et 9 % ne citent pas le document attendu.
- **Hors périmètre, les 12 pièges sont refusés**, proprement, y compris les sujets voisins (Prefect, Dagster, Snowflake). Avec 12 questions, l'intervalle [1 ; 1] ne prouve pas l'absence de défaut : il dit seulement qu'on n'en a pas observé.

> ATTENTION
> **Ces chiffres ne se comparent pas à ceux de la V1** (chapitre 14) : le corpus, l'index, le jeu de questions et le juge ont tous changé. L'answer relevancy passe de 0,581 à 0,926, ce qui est cohérent avec l'hypothèse d'un modèle d'embeddings inadapté en V1, sans la démontrer : il faudrait réévaluer la V2 avec l'ancien modèle, toutes choses égales par ailleurs.

19.4 Analyse des échecs

Le fichier `per_question.csv` permet d'attribuer chaque échec au retrieval ou à la génération, ce qui était l'objectif de la mesure séparée (section 2.2).

Constat	Questions	Origine
Preuve absente des 4 passages	2 sur 73 : `dbt_models_02`, `dbt_models_08`	Retrieval. Le générateur n'y peut rien.
Question restée sans réponse	1 : `dbt_models_08`	Refus justifié : la preuve manquait. Ce n'est pas un faux refus.
Faithfulness inférieure à 0,7	12 sur 72 (50 à 1,0 ; 10 entre 0,7 et 1)	Génération ou bruit du juge : pour 11 des 12, la preuve était bien dans le contexte.
Aucune source, ou pas le document attendu	7 sur 72	Génération : pour 6 des 7, la preuve était dans le contexte.
Faithfulness non calculée	1 : `af_sched_06`	Le juge a renvoyé un JSON non conforme aux deux répétitions ; Ragas a ignoré la question.

Seuil de refus. Le score du meilleur passage sépare bien les deux populations (médiane 7,58 dans le périmètre, −3,54 hors périmètre) mais elles se recouvrent : le seuil suggéré, 1,668, rejette 100 % du hors-périmètre et écarte aussi 3 % des questions légitimes. Comme le modèle refuse déjà seul les 12 pièges, `RELEVANCE_THRESHOLD` reste désactivé : l'activer coûterait des réponses sans rien rapporter.

19.5 Ce que l'exécution réelle a révélé

Quatre défauts sont apparus, alors que les tests passaient et que la CI était verte.

Défaut	Symptôme	Cause	Correction
Import de `_ragas_compat`	*ModuleNotFoundError* après 170 générations, toutes perdues.	Le correctif doit précéder l'import de Ragas ; le tri des imports l'avait placé après.	Import remonté et protégé par `# isort: split`.
Option `--limit`	*IndexError* dans le rapport de seuil.	Quantile calculé sur une liste vide quand aucune question hors périmètre n'est retenue.	Rapport sauté dans ce cas.
Génération des questions	Arrêt au 54e chunk, rien de sauvegardé.	Ollama interrompt une génération qui répète le même token (mode JSON).	Chunk ignoré avec un message ; 2 chunks concernés sur 202.
Filtre anti-fuite	454 questions générées, 61 conservées, dont 7 dans le périmètre.	Seuil cosinus fixe de 0,88, sous le niveau de similarité normal du modèle E5.	Seuil calibré sur le jeu d'évaluation : 0,949.

Le dernier défaut mérite une explication. Avec `multilingual-e5-small`, deux questions distinctes du même domaine sont déjà très proches : dans le jeu d'évaluation, la similarité d'une question avec sa plus proche voisine a une médiane de 0,921, et 86 % dépassent 0,88. Le filtre écartait donc presque tout, y compris 6 des 60 questions hors périmètre rédigées à la main. Le seuil est désormais la plus forte similarité observée entre deux questions d'évaluation : n'est une fuite que ce qui est plus proche d'une

question d'évaluation que ne le sont entre elles les deux plus proches. Résultat : 418 questions conservées, et 35 écartées qui sont de vraies redites.

> LA LEÇON
> Les tests simulaient exactement les parties qui ont cassé : la recherche et le LLM dans le run d'évaluation, les embeddings dans le filtre anti-fuite (`use_embeddings=False`), et Ragas n'était jamais importé. Un test rapide protège la logique ; il ne dit rien des seuils, des versions de bibliothèques ni du comportement d'un vrai modèle. **Un seuil sur une similarité cosinus n'a de sens que pour un modèle d'embeddings donné** : il se calibre sur des données, il ne se recopie pas.

> CE QU'ON AURAIT PU FAIRE
> **Sauvegarder avant chaque étape longue** : écrire les réponses générées avant de lancer Ragas, et permettre à `build_preferences.py` de reprendre où il s'est arrêté. Les deux scripts n'écrivent leur résultat qu'à la fin ; une erreur tardive ou une mise en veille fait tout perdre.

19.6 État du pipeline DPO

Étape	État au 13 octobre 2026	Détail
1 · Questions	**Fait**	454 questions générées ; 418 conservées (358 dans le périmètre, 60 hors périmètre) ; 35 écartées par le filtre anti-fuite, 1 doublon. 5 chunks sans question (2 interrompus, 3 JSON invalides).
2 · Paires de préférences	**Interrompue, à relancer**	Premier run coupé par un redémarrage de Windows après environ 28 questions (16 paires, écarts nets comme 0,94 contre 0,50). Cause de la lenteur : le modèle NLI tournait sur le CPU (environ 45 s par réponse) ; avec PyTorch CUDA, environ 1 s. Le script sait désormais reprendre (section 20.5).
3 · Entraînement QLoRA	À faire	Sur Colab ou Kaggle, si l'étape 2 fournit au moins 200 paires.
4 · Export vers Ollama	À faire	Modèle `rag-assistant-dpo` .
5 · Réévaluation	À faire	Comparaison appariée avec `baseline_v2` .

Ce que la baseline implique. Trois des quatre métriques de la règle de décision sont au plafond : `quotes_faithful` et `citations_valid` valent 1, `false_refusal` vaut 0. Il ne reste de marge que sur `faithfulness` . L'issue la plus probable est donc « pas d'amélioration significative, modèle non adopté », ce qui serait un résultat valide : la règle a été écrite avant de voir les données, précisément pour ne pas adopter un modèle sur une impression. Les deux métriques où une marge existe aussi, `has_citation` et `cites_expected_source` , ne figurent pas dans la règle ; les y ajouter après coup reviendrait à choisir le critère en fonction du résultat.

> ATTENTION
> Les 16 paires du premier run viennent d'une version du script qui ne savait pas reprendre : le prochain lancement demande `--restart` et repart de zéro.

19.7 Prochaines étapes

1. Relancer l'étape 2 (`build_preferences.py --samples 4 --restart`), puis lire `training/data/preferences_stats.json` : nombre de paires, motifs de rejet, questions écartées faute de passage source.
2. Relire 30 paires au hasard : la réponse retenue doit être réellement meilleure, et pas seulement plus courte ou plus évasive.
3. Entraîner sur Colab avec `--limit 20` d'abord, puis le run complet ; exporter et réévaluer avec `--repeats 2` .
4. Appliquer la règle de décision avec `compare_runs.py baseline_v2 dpo_v1` , quel que soit le verdict.
5. Indépendamment du DPO : annoter 30 réponses à la main pour mesurer l'accord du juge, et examiner les deux questions que le retrieval manque.

20 Observabilité, interface et conversation

Ces fonctionnalités ont été ajoutées après la mesure de la baseline, pour observer le système et en faire un outil de travail : on peut suivre chaque requête, tester à la main, poursuivre une conversation et recevoir des mises en garde. Elles ne changent pas le prompt de la réponse principale quand il n'y a pas d'historique : les résultats du chapitre 19 restent valables.

20.1 Observabilité avec Langfuse

MLflow (section 10.5) garde des runs agrégés : une configuration, des moyennes, des fichiers. Il ne montre pas ce qui s'est passé pour *une* question. Langfuse enregistre une **trace** par question : la recherche (passages, scores du reranker), l'appel au LLM (prompt exact, réponse, latence, tokens) et le résultat. C'est l'outil qui permet de répondre à « pourquoi cette réponse ? ».

Origine	Trace	Regroupement et scores
API, interface	`question-rag`	Étiquette api ; session = identifiant de conversation.
`run_eval.py`	`eval:<id>`	Session = nom du run ; métriques déterministes et Ragas attachées comme scores, pour filtrer les réponses en échec.
`build_preferences.py`	`dpo:<id>`	Session par run ; récompenses de la paire comme scores.
`run_conversation_eval.py`	`conversation:<id>`	Session = nom du run ; question reformulée.

Exemple réel d'une trace d'API : `question-rag` contient `recherche` (retriever) puis `génération`, qui contient l'appel `ChatOllama` : modèle llama3.1:8b, 1 324 tokens en entrée, 94 en sortie, 14,9 s.

Fonction de rag_chain/tracing.py	Rôle
`is_active()`	Clés présentes et serveur joignable (vérifié une fois, 3 s au plus).
`observe(name, as_type)`	Décorateur : trace une fonction (recherche, question, compléments).
`langchain_config(run_name)`	Rattache un appel LangChain au LLM à la trace en cours.
`trace(...)`, `attributes(...)`	Ouvre une trace ; étiquette et regroupe en session.
`set_output`, `score`, `score_trace`	Sortie et scores de la trace en cours ou d'une trace terminée (Ragas, après coup).
`current_trace_id`, `trace_url`, `flush`	Identifiant et lien de la trace ; envoi des données en fin de script.

POURQUOI CE CHOIX

Optionnel et sans dépendance cachée. Sans clés, aucune fonction n'agit et `langfuse` n'est pas importé : les tests et la CI ne dépendent d'aucun serveur. La vérification de disponibilité a été ajoutée après un incident : avec des clés mais des conteneurs arrêtés, chaque lot de traces attendait l'expiration de son envoi, et une évaluation restait bloquée plusieurs minutes à la fin. Désormais le traçage se désactive avec un avertissement.

Serveur local. Le dossier `langfuse/` contient le `docker-compose.yml` officiel (Langfuse v4 : interface, worker, Postgres, ClickHouse, Redis, MinIO), un override sans redémarrage automatique, qui fait aussi entrer les nouveaux comptes dans l'organisation du projet, et `init_env.py`, qui génère des secrets aléatoires à la place des mots de passe par défaut (`langfuse/.env`, non versionné). Vérification : `python -m rag_chain.tracing`.

CE QU'ON AURAIT PU FAIRE

Langfuse Cloud : rien à faire tourner, mais les questions et passages partent sur un serveur tiers. **Le traçage MLflow** (`mlflow.langchain.autolog`) : déjà installé, le plus léger, interface moins riche pour les sessions et les scores. **Phoenix** (Arize) ou **OpenTelemetry + Jaeger** : standards ouverts, plus génériques. **LangSmith** : payant au-delà d'un quota et non local.

20.2 Interface web de test

`api/static/index.html` est servie sur `/` par l'API. Elle montre tout ce qu'il faut pour juger une réponse : badge de refus (en distinguant le refus par seuil, sans appel au LLM), sources citées et sources récupérées mais non citées, passages exacts fournis au modèle avec leur score de reranking et leur cosinus, question reformulée, compléments, modèle, latence et lien vers la trace Langfuse.

Champ de l'API	Sens	Rôle
`history`	requête	Échanges précédents {question, answer}, du plus ancien au plus récent.
`conversation_id`	requête	Identifiant de conversation, devient la session Langfuse.
`complements`	requête	Active la recherche de compléments (faux par défaut).
`standalone_question`	réponse	Question reformulée qui a servi à la recherche.
`passages`	réponse	Passages fournis au LLM : source, section, scores, texte, cité ou non.
`complements`, `trace_url`	réponse	Compléments vérifiés ; lien de la trace.

POURQUOI CE CHOIX

Une page statique, sans framework. Pas d'étape de build ni de second processus : elle est servie par l'API, y compris dans l'image Docker. C'est un outil de test pour voir ce que le système a lu et décidé, pas un produit fini.

CE QU'ON AURAIT PU FAIRE

Streamlit ou **Gradio** : rapides à écrire en Python, mais un second serveur. **Chainlit** : spécialisé dans les interfaces de chat pour LLM, avec affichage des étapes intermédiaires. Un front **React** : pour un produit, surdimensionné pour tester.

20.3 Mémoire de conversation

Sans mémoire, chaque question est traitée seule. Une relance comme « Laquelle est utilisée par défaut ? » ne contient pas son sujet : la recherche vectorielle n'a rien sur quoi s'appuyer. La mémoire sert d'abord la **recherche**, puis la génération.

1. L'interface envoie l'historique avec chaque question ; l'API ne stocke rien.
2. `condense_question` demande au LLM de reformuler la relance en question autonome, à partir des `HISTORY_MAX_TURNS` derniers échanges (3 par défaut). Garde-fou : une sortie vide ou trop longue (le modèle a répondu au lieu de reformuler) est remplacée par la question d'origine.
3. La recherche utilise la question reformulée, affichée dans l'interface.
4. La génération reçoit les derniers échanges entre le message système et la question (`MessagesPlaceholder`), les réponses étant tronquées à 1 500 caractères. Les sources restent les passages de la question courante.
5. Sans historique, le prompt est identique au caractère près : évaluation et DPO ne sont pas affectés.

Mesure. `evaluation/conversations.json` contient 18 relances, chacune rattachée à une question du jeu d'évaluation dont les preuves sont connues. L'historique contient la question précédente et sa réponse de référence. On compare la recherche avec la relance brute, reformulée, et avec la question écrite en entier (plafond).

Métrique	Relance brute	Reformulée	Question complète	Écart
`evidence_hit`	0,444 [0,22 ; 0,67]	**0,778 [0,56 ; 0,94]**	1,000 [1,00 ; 1,00]	+0,333
`evidence_recall`	0,444 [0,22 ; 0,67]	**0,750 [0,53 ; 0,94]**	0,972 [0,92 ; 1,00]	+0,306
`mrr`	0,389 [0,19 ; 0,61]	**0,671 [0,46 ; 0,87]**	0,866 [0,74 ; 0,97]	+0,282
`source_hit`	0,722 [0,50 ; 0,94]	**1,000 [1,00 ; 1,00]**	1,000 [1,00 ; 1,00]	+0,278

Les quatre écarts sont significatifs (bootstrap apparié). Sur les 4 relances où la reformulation ne retrouve pas la preuve, une a changé de sens (le modèle a recopié la question précédente) ; les trois autres sont correctes mais formulées autrement que le passage, ce qui montre aussi la sensibilité du

retrieval à la formulation.

POURQUOI CE CHOIX

Historique côté client. L'API reste sans état : rien à stocker, rien à purger, plusieurs instances possibles. L'identifiant de conversation sert seulement à regrouper les traces.

ATTENTION

18 relances et une seule exécution : un ordre de grandeur, pas une mesure fine. L'historique de la mesure utilise les réponses de référence, pas des réponses générées, et l'ancrage des réponses générées avec historique n'est pas encore évalué. La reformulation coûte un appel au LLM par relance.

CE QU'ON AURAIT PU FAIRE

Concaténer les dernières questions à la requête de recherche : gratuit, mais bruité. **Mémoire côté serveur** (sessions, Redis) : nécessaire pour reprendre une conversation plus tard. **Résumé glissant** de la conversation : pour les longues discussions, au-delà de la fenêtre de contexte. **LangGraph** et ses points de sauvegarde : pour des agents à plusieurs étapes.

20.4 Compléments : mises en garde vérifiées (expérimental)

Objectif : signaler ce qu'un ingénieur risque d'ignorer, comme le ferait un collègue (« attention, les colonnes de unique_key ne doivent pas être nulles »). L'ajouter au prompt principal aurait trois défauts : les 4 passages sont choisis pour répondre, pas pour trouver les pièges, donc le modèle puiserait dans sa mémoire ; la faithfulness, l'answer relevancy et la concision de la récompense DPO baisseraient sans qu'on sache si le système s'améliore ; et le prompt unique de l'évaluation et du DPO serait modifié. D'où un module séparé, `rag_chain/complements.py` .

Étape	Ce qui est fait
1 · Recherche dédiée	40 candidats reclassés par le reranker ; ne garde que les passages contenant un encadré (Note, Warning, Tip, Info…, conservés par build_corpus.py) ou issus du guide de bonnes pratiques d'Airflow, absents du contexte de la réponse, de score ≥ 0.
2 · Génération	Un appel au LLM en JSON : au plus 3 compléments typés (mise en garde, autre façon de faire, à savoir), chacun avec son document. Une liste vide est la réponse attendue le plus souvent.
3 · Filtre par programme	Document parmi ces passages ; citations et identifiants de code vérifiés (checks.py) ; pas de question à la place d'une information.
4 · Filtre NLI	Probabilité d'implication moyenne ≥ 0,5 sur les phrases (modèle de la récompense DPO, déplacé dans rag_chain/nli.py). Si le NLI ne se charge pas, rien n'est affiché.

Observé sur 8 questions. Aucun complément pour 5 d'entre elles. Pour les 3 autres, des compléments utiles et tirés du texte (l'avertissement sur `dbt-snowflake` avant la version 1.10.6 pour les modèles

incrémentaux, les self-checks et l'environnement de test pour les Dags), mais aussi des platitudes ancrées (« il est important de connaître la structure du Dag »). Coût : 5 à 10 s par question. Avant le filtre NLI, le modèle produisait des conseils génériques présentés comme tirés de la documentation.

> ATTENTION
> Le NLI vérifie qu'une phrase découle d'un passage, pas qu'elle est utile ni liée à la question. Les seuils (score du reranker ≥ 0, implication ≥ 0,5) ne sont pas calibrés et aucune mesure n'existe encore : la fonction reste expérimentale, et désactivée par défaut dans l'API.

> CE QU'ON AURAIT PU FAIRE
> **Mesurer d'abord** : un jeu de questions dont la mise en garde attendue est connue (rappel des mises en garde, précision des compléments), comme pour la mémoire. **Questions de suivi cliquables** : moins risqué, aucune nouvelle affirmation. **Marquer les mises en garde à l'indexation** (métadonnée du chunk) plutôt qu'à la volée. **Juge d'utilité** par LLM : utile pour trier, mais bruité.

20.5 Robustesse du pipeline DPO

Le goulot était le NLI, pas le LLM. Un chronométrage étape par étape a montré qu'une génération prenait environ 11 s et la notation d'une réponse de 31 mots environ 45 s : PyTorch était installé sans CUDA et le modèle NLI tournait sur le processeur. Avec la roue CUDA de PyTorch, la notation prend environ 1 s sur une RTX 4060. `build_preferences.py` affiche maintenant le périphérique du NLI au démarrage.

Reprise après interruption. Le script écrit sa progression question par question (`preferences_progress.jsonl`) et chaque paire immédiatement. Relancé, il reprend où il s'était arrêté et retire une paire écrite juste avant l'interruption, pour ne pas l'avoir en double. Il refuse de reprendre si les paramètres ou les questions ont changé (`--restart` repart de zéro). Quatre tests couvrent ces cas (`tests/test_preferences_resume.py`).

> LA LEÇON
> Le premier run de nuit a été coupé à 3 h 16 par un redémarrage de Windows Update, après environ 28 questions sur 418 : tout était perdu. Un traitement de plusieurs heures doit sauvegarder au fil de l'eau. Et avant d'optimiser, mesurer : l'intuition désignait le LLM, le chronométrage a désigné le NLI.

ANNEXES

Glossaire et révision

Les termes techniques du document, puis des questions de révision corrigées pour vérifier sa compréhension du projet.

A Glossaire

Terme	Définition
Adaptateur (LoRA)	Petit ensemble de poids appris qui modifie un modèle gelé ; s'applique au modèle de base ou s'y fusionne.
ANN	Approximate Nearest Neighbors : recherche approchée des plus proches voisins.
Answer relevancy	Métrique Ragas : la réponse répond-elle à la question posée ? Mesurée via des questions régénérées.
Bi-encodeur	Modèle qui encode séparément la question et le passage ; permet l'indexation préalable.
Bootstrap	Rééchantillonnage avec remise pour estimer la variabilité d'une statistique.
BM25	Score de recherche lexicale fondé sur la fréquence des termes ; référence des moteurs de recherche classiques.
Chunk	Passage de document indexé et récupéré comme une unité.
Context precision	Métrique Ragas : les passages utiles sont-ils bien classés en tête ?
Cross-encodeur	Modèle qui lit ensemble la question et le passage pour noter leur pertinence ; utilisé en reranking.
DPO	Direct Preference Optimization : alignement sur des paires de préférences par une perte de classification, sans modèle de récompense explicite.
Embedding	Vecteur qui représente le sens d'un texte.
Faithfulness	Métrique Ragas : part des affirmations de la réponse étayées par le contexte.
Fil d'Ariane	Chemin de titres (« Document > Section > Sous-section ») placé en tête de chaque chunk.
GGUF, Q4_K_M	Format de modèle de llama.cpp / Ollama ; Q4_K_M est une quantification à environ 4 bits.
GHCR	GitHub Container Registry, registre d'images Docker lié au dépôt.
Grounding	Ancrage : la réponse découle-t-elle des sources ? Dans la récompense, mesuré par NLI.
HNSW	Graphe hiérarchique utilisé par ChromaDB pour la recherche approchée.
Hallucination	Affirmation plausible mais fausse ou non étayée produite par un LLM.
Intervalle de confiance	Plage plausible pour la vraie valeur d'une métrique, compte tenu de l'échantillon de questions.
LCEL	LangChain Expression Language : composition de composants par l'opérateur `|`.
MDX	Markdown enrichi de composants JSX/React, format du site de dbt.
MRR	Mean Reciprocal Rank : moyenne de 1 / rang du premier résultat pertinent.
NF4	Format de quantification 4 bits de QLoRA, adapté à la distribution des poids.
NLI	Natural Language Inference : implication, contradiction ou neutralité entre deux textes.
On-policy	Données produites par le modèle qu'on entraîne lui-même.
QLoRA	LoRA appliqué à un modèle chargé en 4 bits.
RAG	Retrieval-Augmented Generation : génération à partir de passages récupérés.
Reranking	Reclassement fin d'une liste de candidats, ici par un cross-encodeur.
Reward hacking	Le modèle optimise les failles de la récompense au lieu de l'objectif réel.
RLHF, PPO	Apprentissage par renforcement à partir de préférences humaines ; PPO est l'algorithme de renforcement classiquement utilisé.
RST	reStructuredText, format de la documentation d'Airflow (Sphinx).

Terme	Définition
Test apparié	Comparaison de deux versions sur les mêmes questions, à partir des différences question par question.
Token	Unité de texte manipulée par un modèle (morceau de mot).
TRL	Bibliothèque de Hugging Face pour l'apprentissage par préférences et par renforcement des LLM.

B Questions de révision

Essaie de répondre avant de lire la correction.

1. Pourquoi mesurer le retrieval séparément de la génération ?

Parce qu'un RAG a deux points de défaillance. Une mauvaise réponse peut venir d'un passage non récupéré (le générateur n'y peut rien) ou d'un bon passage mal exploité. Sans mesure séparée, on ne sait pas quoi corriger. Le projet mesure le retrieval sans LLM, grâce aux preuves.

2. Pourquoi le modèle d'embeddings de la V1 posait-il problème ?

Il était entraîné sur de l'anglais alors que les questions sont en français et le corpus en anglais. La recherche cross-lingue n'était pas maîtrisée, et Ragas utilisait le même modèle pour calculer answer_relevancy.

3. À quoi servent les préfixes « query: » et « passage: » ?

Les modèles E5 ont été entraînés avec eux : ils indiquent si le texte est une question ou un document. Les omettre dégrade la qualité de la recherche.

4. Pourquoi deux étapes de recherche ?

Le bi-encodeur trie rapidement tout l'index mais grossièrement ; le cross-encodeur est précis mais trop lent pour tout l'index. On combine un rappel large (20 candidats) et un tri fin (4 retenus).

5. Que se passerait-il si l'on changeait de modèle d'embeddings sans réindexer, et comment le projet l'empêche-t-il ?

Les vecteurs des questions et des passages viendraient de deux espaces différents : résultats absurdes, sans erreur. `ensure_index_ready` compare le modèle enregistré dans l'index à celui configuré et lève `IndexNotReadyError` ; l'API réindexe au démarrage si l'index est obsolète.

6. Pourquoi mesurer la taille des chunks avec le tokenizer d'embeddings ?

Parce que c'est lui qui limite l'encodage (512 tokens pour E5) : au-delà, la fin du chunk est ignorée dans le vecteur. Un autre tokenizer donne des comptes différents.

7. Un écart de 0,07 de faithfulness entre deux configurations évaluées sur 15 questions est-il significatif ?

Non. Pour une métrique de l'ordre de 0,8 sur 15 questions, l'écart-type de la moyenne est d'environ 0,10, soit un intervalle d'environ ±0,2. Il faut plus de questions, un test apparié et idéalement des répétitions.

8. Pourquoi un test apparié est-il plus sensible qu'une comparaison de deux moyennes ?

Les deux versions sont évaluées sur les mêmes questions ; la difficulté propre à chaque question affecte les deux de la même façon et disparaît dans la différence. La variance restante est bien plus faible.

9. Qu'est-ce qu'un faux refus, et pourquoi ne compte-t-on que ceux où la preuve était dans le contexte ?

Un refus à une question du périmètre. Si la preuve n'avait pas été récupérée, refuser était le bon comportement : l'erreur vient du retrieval, pas du générateur.

10. Pourquoi exclure les questions hors périmètre de Ragas ?

Un refus correct n'a pas d'affirmations à vérifier : Ragas lui donne 0, ce qui pénalise le bon comportement et fait baisser les moyennes. Le refus est mesuré par `correct_refusal`.

11. Pourquoi utiliser un juge différent du générateur ?

Pour limiter le biais d'auto-évaluation : un modèle tend à juger favorablement ses propres formulations.

12. Quelle est la différence entre DPO et PPO ?

PPO entraîne d'abord un modèle de récompense puis optimise le LLM contre lui par renforcement, en générant pendant l'entraînement. DPO optimise directement une perte de classification sur des paires fixes, sans modèle de récompense ni génération pendant l'entraînement : plus simple et plus stable, mais limité aux paires disponibles.

13. Que représente β dans la perte DPO ?

L'intensité de la contrainte qui retient le modèle près du modèle de référence. Plus β est grand, moins le modèle s'en écarte.

14. Pourquoi la récompense n'utilise-t-elle pas un juge LLM ?

Un juge de 7-8B est bruité et biaisé ; optimiser contre lui pousse le modèle à exploiter ses failles (reward hacking). Les contraintes dures sont vérifiées par programme et ne peuvent pas être trompées ; le grounding utilise un modèle NLI dédié.

15. Pourquoi écarter une question d'entraînement dont le chunk source n'a pas été récupéré ?

On ne saurait pas si un refus serait correct : la paire serait ambiguë et pourrait apprendre au modèle à répondre sans source.

16. À quoi sert le filtre anti-fuite ?

À garantir que les questions d'entraînement ne sont pas des quasi-copies des questions d'évaluation. Sinon, une amélioration après DPO pourrait refléter de la mémorisation et non une meilleure capacité.

17. Pourquoi le gabarit de chat d'entraînement reproduit-il celui d'Ollama plutôt que celui de Hugging Face ?

Le modèle doit être entraîné sur le format exact qu'il recevra en production. Le gabarit Hugging Face de Llama 3.1 ajoute des lignes de date au message système ; Ollama non.

18. Pourquoi l'entraînement DPO ne peut-il pas tourner dans la CI ?

Depuis la version 1.15, TRL calcule les log-probabilités avec un noyau Triton, qui exige un GPU CUDA. Les runners gratuits de GitHub n'ont pas de GPU ; le test CPU vérifie tout ce qui précède l'entraînement.

19. Que vérifie le job CI « Corpus » ?

Qu'en reconstruisant le corpus depuis les sources figées, on obtient exactement les fichiers versionnés : le corpus est reproductible et n'a pas été modifié à la main.

20. Quel est le premier travail à faire avant de lancer le DPO, et pourquoi ?

Mesurer la baseline V2. Si les métriques que le DPO vise (citations, faux refus, faithfulness) sont déjà au plafond, l'entraînement n'a rien à corriger ; sinon, la baseline sert de référence pour la comparaison appariée.